



Fachhochschule der Wirtschaft
FHDW Bielefeld

Studienarbeit im Modul Classification and Clustering

Kundensegmentierung mit RetailRocket und Online Retail II

Vorgestellt von:

Christian Roth

Senner Straße 68

33647 Bielefeld

`christian.roth@edu.fhdw.de`

Studiengang:

Wirtschaftsinformatik mit Schwerpunkt Data Science (*M.Sc.*)

Prüferin:

Prof. Dr. Yvonne Gorniak

Eingereicht am:

30. September 2025 in Bielefeld

Gendererklärung

Aus Gründen der besseren Lesbarkeit wird auf die gleichzeitige Verwendung der Sprachformen männlich, weiblich und divers (m/w/d) verzichtet. Sämtliche Personenbezeichnungen gelten gleichermaßen für alle Geschlechter.

Abstract

Kunden im E-Commerce verhalten sich sehr unterschiedlich in ihrer Kaufhäufigkeit, ihren Ausgaben und ihrer Reaktion auf Kampagnen. Für Unternehmen liegt darin eine große Chance, denn differenzierte Kundensegmente ermöglichen eine gezieltere Ansprache, nachhaltigere Bindung und wirksamere Sortimentsentscheidungen. Gleichzeitig stellt diese Vielfalt eine Herausforderung dar, da sich Strukturen in den Daten nicht unmittelbar erschließen und klassische Auswertungen oft oberflächlich bleiben.

Diese Arbeit möchte die Forschungsfrage beantworten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie diese so beschrieben werden können, dass Marketing und CRM sie unmittelbar nutzen können. Clustering wird dabei nicht als Selbstzweck verstanden, sondern als Werkzeug, das Datenmuster sichtbar macht und in verständliche Profile übersetzt.

Das Vorgehen orientiert sich am CRISP-DM-Modell. Durch Feature Engineering werden Kennzahlen wie Recency, Frequency und Monetary Value sowie Warenkorb- und Sortimentsmerkmale gebildet. Die Segmentierung erfolgt mit K-Means für kompakte Strukturen und HDBSCAN für dichte-basierte Cluster. Die Qualität wird mit dem Silhouette-Koeffizienten und dem DBCV-Index bewertet. Die Hopkins-Statistik wird ergänzend zur Prüfung der Clustertendenz eingesetzt, ihre Aussagekraft für hochdimensionale, sparse Daten jedoch kritisch eingeordnet.

Die Ergebnisse zeigen mehrere unterscheidbare Segmente, die durch Kennzahlentabellen, Personas und Visualisierungen anschaulich dargestellt werden. Damit können Fachbereiche die Segmente für Kampagnen, Kundenbindungsprogramme und Sortimentsentscheidungen einsetzen. Die Arbeit zeigt, dass sich Segmentstrukturen in E-Commerce-Daten identifizieren, verständlich aufbereiten und in handlungsleitende Marketingmaßnahmen überführen lassen. Ein Ausblick skizziert die Ergänzung durch externe Validierung, Text-Mining und dynamische Segmentierungen.

Inhaltsverzeichnis

Gendererklärung	i
Abstract	ii
Inhaltsverzeichnis	iii
Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
1 Einleitung	5
1.1 Problemstellung	5
1.2 Zielsetzung	5
1.3 Vorgehensweise	6
2 Theoretischer Hintergrund	7
2.1 Kundensegmentierung im E-Commerce	7
2.2 Clustering als Mittel zum Zweck	7
3 Datenbasis und Aufbereitung	9
3.1 Roadmap der Segmentierung im Projekt	9
3.2 Datensätze Online Retail II / RetailRocket	10
4 Methodik (CRISP-DM: Modeling - Theoretische Grundlagen)	11
4.1 Feature Engineering und Preprocessing (RFM, Warenkorbgrößen, Sortiments- merkmale – konzeptionell)	11
4.2 Clustering-Verfahren (K-Means, HDBSCAN - zentrale Eigenschaften) . . .	13
4.3 Clustertendenz und Modelle Hopkins, K-Means, HDBSCAN	13
4.4 Validierung (Hopkins, Silhouette, DBCV) als Qualitätskontrolle	14
5 Ergebnisse und Diskussion (CRISP-DM: Modeling und Evaluation - praktisch)	16

5.1	Preprocessing (Bereinigung, Skalierung, Variablenableitung – Resultate, Details in Anhang C)	16
5.2	Clustering (K-Means und HDBSCAN – Ergebnisse und Validierung)	16
5.3	Interpretation der Segmente (Clusterprofile, Personas, Visualisierungen) . .	17
6	Diskussion	18
6.1	Vergleich der Verfahren	18
6.2	Implikationen für Marketing und CRM	18
6.3	Grenzen und methodische Limitationen	18
7	Fazit und Ausblick	19
7.1	Beantwortung der Forschungsfrage	19
7.2	Limitationen	19
7.3	Ausblick (z. B. Text Mining, Echtzeit-Segmentierung)	20
	Literaturverzeichnis	21
	Anhang	22
	Anhangsverzeichnis	22
	Anhang A: Prozessmodelle	23
	A.1: Das CRISP-DM-Modell	24
	A.2: Segmentation Management Strategy	25
	Anhang B: Clustering und RFM	27
	B.1: Eindimensionale RFM-Segmentierung	28
	B.2: Mehrdimensionale RFM-Segmentierung	28
	B.3: Retail-Segmentierung mit RFM	29
	B.4: Warenkorb- und Sortimentsmerkmale	33
	Anhang C: Datenaufbereitung und Voranalysen	47
	C.1 Variablenbeschreibung und Quellen	48
	C.2: Abbildung auf ein einheitliches Ereignisschema	52
	C.3: Bereinigungsschritte	56
	C.4: Normalisierung der Zeitstempel	59
	C.5: Zusammenfassung des bereinigten Datenbestands	62
	C.6 Clustertendenz mit der Hopkins-Statistik	65
	C.7 Modellierungsschritte mit Python: K-Means und HDBSCAN	69
	Glossar	76

Abbildungsverzeichnis

3.1	Eigene Darstellung der Segmentierungs-Roadmap mit Transformation von Rohdaten über Cluster und Segmente hin zu strategischen Maßnahmen . .	10
A.1	Das CRISP-DM-Modell nach Wirth und Hipp (2000), S. 5	24
A.2	Roadmap der Segmentation Management Strategy nach Tsiptsis und Chorianopoulos (2010), S. 213–216	25
A.3	Beispiel für eindimensionale RFM-Einteilungen in fünf Quantile je Dimension. Hervorgehoben ist die RFM-Zelle 351 (Recency = 3, Frequency = 5, Monetary = 1) nach Tsiptsis und Chorianopoulos (2010), S. 336.	28
A.4	Eigene Darstellung eines RFM-Würfels	32
A.5	Warenkorbanalyse verschiedener Kundentypen mit Durchschnittswerten, Bestellhäufigkeit und Vergleichsdarstellung	39
A.6	Sortimentsverhalten: Vergleich von Generalisten (gleichmäßige Diversifikation) und Spezialisten (dominante Kategorie)	46

Tabellenverzeichnis

7.1	Übersicht der Variablen in beiden Datenquellen	51
7.2	Abbildung der Variablen auf das Ereignisschema	52

```

#| eval=TRUE
#| message=FALSE
#| warning=FALSE
#| include=FALSE

# CRAN-Mirror setzen (WICHTIG: Vor allen anderen Operationen)
options(repos = c(CRAN = "https://cran.rstudio.com/"))

# Arbeitsumgebung leeren (alle Objekte entfernen)
rm(list = ls())
gc()

```

 Terminal-Output

	used (Mb)	gc trigger (Mb)	max used (Mb)
Ncells	597608 32.0	1347142 72	704004 37.6
Vcells	1138677 8.7	8388608 64	1878412 14.4

```

# Root für Quarto setzen
knitr::opts_knit$set(root.dir = here::here())

# Installiere und lade erforderliche Bibliotheken
if (!requireNamespace("reticulate", quietly = TRUE))
install.packages("reticulate")
if (!requireNamespace("rmarkdown", quietly = TRUE))
install.packages("rmarkdown")
if (!requireNamespace("shiny", quietly = TRUE)) install.packages("shiny")
if (!requireNamespace("dplyr", quietly = TRUE)) install.packages("dplyr")
if (!requireNamespace("tidyr", quietly = TRUE)) install.packages("tidyr")
if (!requireNamespace("tidyverse", quietly = TRUE))
install.packages("tidyverse")
if (!requireNamespace("forecast", quietly = TRUE))
install.packages("forecast")
if (!requireNamespace("ggpubr", quietly = TRUE)) install.packages("ggpubr")
if (!requireNamespace("viridis", quietly = TRUE)) install.packages("viridis")

```



```
if (!requireNamespace("naniar", quietly = TRUE)) install.packages("naniar")
if (!requireNamespace("GGally", quietly = TRUE)) install.packages("GGally")
if (!requireNamespace("corrplot", quietly = TRUE))
install.packages("corrplot")
if (!requireNamespace("kableExtra", quietly = TRUE))
install.packages("kableExtra")
if (!requireNamespace("gt", quietly = TRUE)) install.packages("gt")
if (!requireNamespace("grid", quietly = TRUE)) install.packages("grid")
if (!requireNamespace("gridExtra", quietly = TRUE))
install.packages("gridExtra")
if (!requireNamespace("patchwork", quietly = TRUE))
install.packages("patchwork")

library(reticulate)
library(rmarkdown)
library(shiny)
library(dplyr)
library(tidyr)
library(tidyverse)
library(forecast)
library(ggpubr)
library(viridis)
library(naniar)
library(GGally)
library(kableExtra)
library(gt)
library(grid)
library(gridExtra)
library(patchwork)
```

```
## eval=TRUE
## message=FALSE
## warning=FALSE
```

```
#!/ include=FALSE

# Erforderliche Bibliotheken installieren (falls nicht vorhanden)
import subprocess
import sys

def install_and_import(package_name, import_name=None):
    """
    Installiert ein Paket und importiert es.
    package_name: Name für pip install
    import_name: Name für import (falls unterschiedlich)
    """
    if import_name is None:
        import_name = package_name

    try:
        __import__(import_name)
    except ImportError:
        print(f"Installing {package_name}...")
        subprocess.check_call([sys.executable, "-m", "pip", "install",
package_name])
    finally:
        globals()[import_name] = __import__(import_name)

# Bibliotheken mit korrekten Namen installieren
packages = [
    ("pandas", "pandas"),
    ("numpy", "numpy"),
    ("matplotlib", "matplotlib"),
    ("seaborn", "seaborn"),
    ("scipy", "scipy"),
    ("statsmodels", "statsmodels"),
    ("pulp", "pulp"),
    ("scikit-learn", "sklearn"), # Hier ist der Unterschied!
]
```

```
    ("hdbscan", "hdbscan")
]

for pkg_name, import_name in packages:
    install_and_import(pkg_name, import_name)

# Bibliotheken laden
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.tsa.seasonal import STL
from pulp import LpMaximize, LpProblem, LpVariable, value

# Scikit-learn Imports hinzufügen
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.utils.validation import check_array

# HDBSCAN Import mit Fehlerbehandlung
try:
    import hdbscan
except ImportError:
    print("Installing hdbscan...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "hdbscan"])
    import hdbscan

print("Alle Bibliotheken erfolgreich geladen!")
```

Terminal-Output

Alle Bibliotheken erfolgreich geladen!

1 Einleitung

1.1 Problemstellung

Der Onlinehandel hat sich zu einem der wichtigsten Vertriebskanäle entwickelt. Millionen von Kunden kaufen regelmäßig ein, doch ihre Verhaltensweisen unterscheiden sich erheblich. Manche Kunden tätigen häufig kleine Käufe, andere bestellen selten, geben dabei aber hohe Summen aus. Hinzu kommen Unterschiede in der Sortimentswahl, im Preisbewusstsein und in der Reaktion auf Kampagnen.

Für Unternehmen liegt darin einerseits eine große Chance, da differenzierte Segmente eine gezielte Ansprache und Bindung ermöglichen. Andererseits stellt diese Vielfalt eine Herausforderung dar. Klassische Auswertungen wie Umsatzstatistiken oder Produktlisten zeigen nur an der Oberfläche Unterschiede, erfassen aber nicht die zugrunde liegenden Strukturen. Viele Segmentierungsprojekte scheitern daran, dass sie entweder methodisch zu stark auf Verfahren fokussieren oder dass die Ergebnisse nicht in verständliche, umsetzbare Formate übersetzt werden.

1.2 Zielsetzung

Die Arbeit möchte die Forschungsfrage beantworten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie diese so dargestellt werden können, dass sie für Marketing und CRM unmittelbar nutzbar sind. Mit den Segmentprofilen, den Personas und den daraus abgeleiteten Maßnahmen wird gezeigt, wie Datenmuster in klare Handlungsoptionen übersetzt werden können. Clustering wird dabei als Werkzeug verstanden, nicht als Selbstzweck.

Der Beitrag dieser Arbeit liegt in drei Aspekten. Erstens wird ein nachvollziehbarer

Prozess von der Datenbasis über die Merkmalsbildung bis zur Segmentierung entwickelt. Zweitens werden Segmentstrukturen durch Kennzahlen, Visualisierungen und narrative Personas anschaulich aufbereitet. Drittens werden praxisrelevante Maßnahmen für Marketing und Kundenbindung abgeleitet.

1.3 Vorgehensweise

Das Vorgehen orientiert sich am CRISP-DM-Modell. Zunächst werden theoretische Grundlagen der Kundensegmentierung und die Rolle von Validierungsmaßen erläutert. Anschließend werden die verwendeten Datensätze beschrieben und durch Feature Engineering in aussagekräftige Merkmale transformiert, unter anderem Recency, Frequency, Monetary Value sowie Warenkorb- und Sortimentsvariablen.

Die Segmentierung wird mit zwei Verfahren durchgeführt: K-Means dient als baseline für kompakte Cluster, während HDBSCAN unregelmäßige Strukturen und Rauschen abbilden kann. Die Qualität wird mit Silhouette und DBCV überprüft. Die Hopkins-Statistik wird ergänzend eingesetzt, wobei ihre eingeschränkte Aussagekraft für hochdimensionale Transaktionsdaten kritisch eingeordnet wird.

Im Ergebnisteil werden die Segmente dargestellt, mit Kennzahlen beschrieben und durch Visualisierungen anschaulich gemacht. Personas übersetzen die Ergebnisse in leicht verständliche Profile. Anschließend werden Implikationen für Marketing und CRM diskutiert. Die Arbeit schließt mit einer Reflexion der Ergebnisse, Limitationen und einem Ausblick auf zukünftige Erweiterungen.

2 Theoretischer Hintergrund

2.1 Kundensegmentierung im E-Commerce

Kundensegmentierung ist ein zentrales Instrument im Customer Relationship Management (CRM), um heterogene Kundenmengen in homogene Gruppen zu unterteilen und diese gezielt anzusprechen. Tsipstis und Chorianopoulos (2010), S. 39 betonen: “Clustering techniques identify meaningful natural groupings of records and group customers into distinct segments with internal cohesion.” Ziel ist die Entwicklung unterscheidbarer Kundentypologien, die „transparent, meaningful, and actionable“ sind (Tsipstis und Chorianopoulos (2010), S. 129).

Die Bedeutung von Kundensegmentierung im E-Commerce hat mit der Verfügbarkeit umfangreicher Transaktions- und Interaktionsdaten deutlich zugenommen. Personalisierte Angebote, zielgerichtete Kampagnen und differenzierte Kundenbindungsstrategien setzen voraus, dass heterogene Kundenmengen in konsistente Gruppen überführt werden können, die sich anhand ihres Verhaltens und Werts klar unterscheiden.

2.2 Clustering als Mittel zum Zweck

In dieser Arbeit wird Clustering nicht als Selbstzweck betrachtet, sondern als Werkzeug, um verborgene Muster zu erkennen und in verständliche Kundensegmente zu übersetzen. Zwei Verfahren stehen dabei im Mittelpunkt: K-Means und HDBSCAN.

K-Means basiert auf der Idee, die Varianz innerhalb von Clustern iterativ zu reduzieren. Es „starts with an initial cluster solution which is updated and adjusted until no further refinement is possible ... Each iteration refines the solution by reducing the within-cluster variation“ (Tsipstis und Chorianopoulos (2010), S. 85). Jedes Cluster wird dabei durch sein

Zentrum (Zentroid) repräsentiert, und die Anzahl der Cluster muss vorab festgelegt werden. Dieses Verfahren eignet sich besonders für kompakte, globuläre Strukturen.

HDBSCAN hingegen ist eine Weiterentwicklung dichtebasierter Verfahren. Dichtebasierte Ansätze identifizieren Gruppen in Regionen hoher Punktdichte und weisen spärlich besetzte Bereiche als Rauschen aus; DBSCAN nutzt dafür die Parameter ε und MinPts (Chakraborty et al. (2022), S. 87–88). Damit lassen sich auch nicht-konvexe Formen abbilden und Ausreißer explizit behandeln. K-Means eignet sich dagegen vor allem für kompakte, annähernd kugelförmige Strukturen, während hierarchische bzw. dichtebasierte Verfahren arbiträre Formen robuster erfassen können (Chakraborty et al. (2022), S. 96). HDBSCAN kombiniert diese Eigenschaften, indem es auf einer hierarchischen Betrachtung lokaler Dichten aufbaut und dadurch stabilere Cluster extrahieren kann.

Der gesamte Prozess wird im Rahmen von CRISP-DM strukturiert. Wirth und Hipp (2000), S. 4 beschreiben: „The life cycle of a data mining project is broken down in six phases“, die von Business Understanding über Data Understanding und Data Preparation bis zu Modeling, Evaluation und Deployment reichen. Diese Phasen sind nicht linear, sondern hochgradig iterativ: “it is never the case that a phase is completely done before the subsequent phase starts” (Wirth und Hipp (2000), S. 9). Eine ausführliche Darstellung und die konkrete Anwendung auf diese Arbeit finden sich in A.1: CRISP-DM-Modell.

3 Datenbasis und Aufbereitung

3.1 Roadmap der Segmentierung im Projekt

Die Abbildung Abbildung 3.1 veranschaulicht den konzeptionellen Ablauf der Segmentierung im Rahmen dieser Studie. Ausgangspunkt sind zahlreiche einzelne Beobachtungen, die in den Datensätzen Online Retail II und RetailRocket vorliegen. Diese Vielzahl an Transaktionen, Warenkörben und Interaktionen erscheint zunächst unstrukturiert und komplex.

Im nächsten Schritt werden diese Daten mithilfe von Clustering-Verfahren wie K-Means und HDBSCAN in Gruppen überführt, die gemeinsame Muster erkennen lassen. Die entstehenden Cluster verdichten sich zu interpretierbaren Segmenten, die durch Kennzahlen, Profile und Personas beschrieben werden können. Dieser Übergang macht sichtbar, dass Segmentierung nicht nur eine rechnerische Klassifikation ist, sondern eine inhaltliche Reduktion von Datenkomplexität in klar unterscheidbare Kundengruppen.

Der abschließende Teil der Abbildung symbolisiert die strategische Nutzung dieser Segmente. Auf Basis der erarbeiteten Kundengruppen können konkrete Maßnahmen für Marketing und CRM entwickelt werden. Damit endet der Prozess nicht bei der Modellierung, sondern wird in eine übergeordnete Strategie überführt, die operative Entscheidungen und Kampagnen unterstützt. Die Visualisierung verdeutlicht somit die Transformation von Rohdaten über Cluster und Segmente hin zu strategischen Maßnahmen und bildet den roten Faden für den weiteren praktischen Teil der Arbeit.

Die dargestellte Roadmap knüpft an das in A.2: Segmentation Management Strategy erläuterte Referenzmodell an. Sie macht deutlich, dass der Segmentierungsprozess als Abfolge von Transformationen zu verstehen ist, die von der Datenbasis über die Modellierung bis zur strategischen Umsetzung führen.

Im Anschluss an die Beschreibung der Datensätze erfolgt die Merkmalsbildung durch

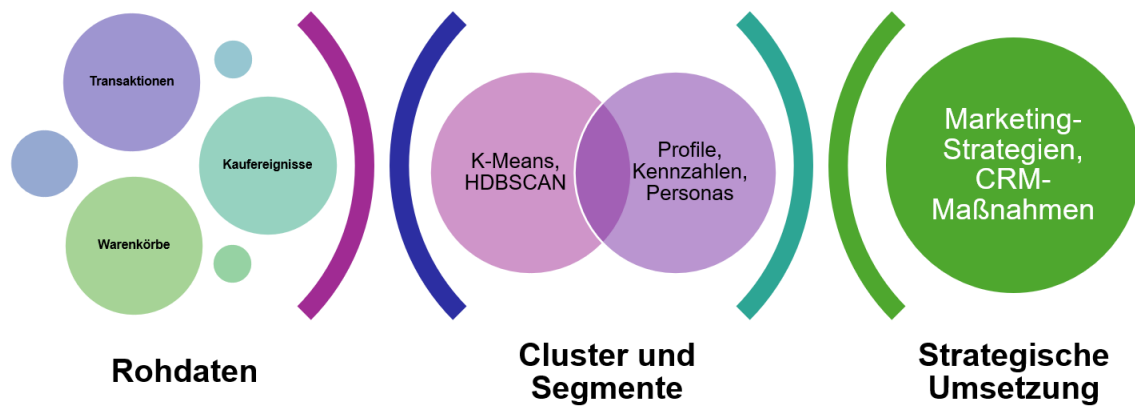


Abbildung 3.1: Eigene Darstellung der Segmentierungs-Roadmap mit Transformation von Rohdaten über Cluster und Segmente hin zu strategischen Maßnahmen

Feature Engineering. Dabei werden aus den Rohdaten zentrale Kennzahlen wie Recency, Frequency, Monetary Value sowie warenkorb- und sortimentsbezogene Variablen abgeleitet. Erst durch diese Konstrukte wird es möglich, im nächsten Schritt Clustermodelle anzuwenden.

Schließlich wird im dritten Teil die Datenaufbereitung behandelt, die für die Robustheit der Modelle entscheidend ist. Dazu gehören die Bereinigung von Ausreißern, die Handhabung fehlender Werte sowie die Skalierung der Merkmale. Mit diesen vorbereitenden Schritten wird die Basis geschaffen, auf der die in der Roadmap dargestellte Transformation technisch und analytisch realisiert werden kann.

3.2 Datensätze Online Retail II / RetailRocket

Um die Forschungsfrage zu beantworten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie diese darstellbar sind, werden zwei komplementäre Datensätze genutzt. RetailRocket erfasst ereignisbasierte Interaktionen wie Ansichten, Warenkorbaktionen und Transaktionen (Zykov et al., 2022). Online Retail II hingegen stellt transaktionsbasierte Rechnungsdaten mit Artikelnummern, Mengen, Preisen und Zeitstempeln bereit (*Datasets - UCI Machine Learning Repository*, 2019).

Die Kombination beider Perspektiven erlaubt es, sowohl verhaltensnahe Muster (RetailRocket) als auch wertorientierte Strukturen (Online Retail II) zu identifizieren. Damit wird die Grundlage gelegt, um robuste und vielseitige Segmentstrukturen zu erschließen.

4 Methodik (CRISP-DM: Modeling - Theoretische Grundlagen)

4.1 Feature Engineering und Preprocessing (RFM, Warenkorbgrößen, Sortimentsmerkmale – konzeptionell)

Damit Segmentstrukturen in den Daten sichtbar werden, müssen aus den Rohdaten aussagekräftige Merkmale abgeleitet werden. Dieser Schritt wird als Feature Engineering bezeichnet (Tsiptsis und Chorianopoulos, 2010, S. 337).

Im Zentrum stehen die klassischen RFM-Variablen, die drei grundlegende Dimensionen des Kaufverhaltens beschreiben:

- *Recency* erfasst den zeitlichen Abstand seit dem letzten Kauf,
- *Frequency* gibt die Anzahl der Käufe in einem bestimmten Zeitraum wieder,
- *Monetary* misst den Umsatz im selben Zeitraum.

Durch die Kombination dieser Dimensionen können Kunden in Zellen eingeteilt werden, die Verhalten und Wert systematisch abbilden. So steht etwa die Zelle 555 für besonders wertvolle Kunden (jüngster Kauf, hohe Frequenz, hoher Umsatz), während 151 Kunden mit langer Kaufpause, geringer Frequenz und geringem Umsatz beschreibt (Tsiptsis und Chorianopoulos, 2010, S. 334–338). Diese Zellenlogik ist leicht verständlich, skaliert aber schlecht, da mit Quintilen bis zu 125 Klassen entstehen können.

Um diese Komplexität zu reduzieren, lässt sich RFM in eine kontinuierliche Kennzahl überführen. Dabei werden die drei Dimensionen zu einem gewichteten Summenscore zusammengefasst:

$$\text{RFM-Score} = w_R \cdot R + w_F \cdot F + w_M \cdot M, \quad w_R + w_F + w_M = 1.$$

Der Vorteil dieser Verdichtung liegt in der besseren Vergleichbarkeit: Statt einer Vielzahl von RFM-Zellen entsteht eine eindimensionale Metrik, die sich leicht sortieren, gruppieren und auch als Input für Clusterverfahren verwenden lässt.

Die Gewichtung der Parameter ist flexibel gestaltbar. Unternehmen können etwa den zeitlichen Abstand (Recency) stärker gewichten, wenn Reaktivierungskampagnen im Vordergrund stehen, oder die Häufigkeit (Frequency), wenn Loyalität im Fokus liegt. Durch Variation der Gewichte w_R , w_F und w_M wird unmittelbar sichtbar, wie sich die Kundenklassifikation verschiebt (Tsiptsis und Chorianopoulos, 2010, S. 338).

Eine anschauliche Visualisierung der RFM-Zellen und ihrer logischen Struktur findet sich in Anhang B.3: Retail-Segmentierung mit RFM.

Ergänzend zur klassischen RFM-Analyse werden in der Literatur weitere behaviorale Variablen vorgeschlagen, um das Kundenverhalten differenzierter zu erfassen. Besonders relevant sind *Warenkorbgrößen* und *Sortimentsmerkmale* (Tsiptsis und Chorianopoulos, 2010, S. 334).

Warenkorbgrößen (*Basket Size*) beschreiben, wie umfangreich einzelne Bestellungen typischerweise ausfallen. Sie können entweder über die durchschnittliche Anzahl enthaltener Artikel oder über den durchschnittlichen Bestellwert erfasst werden. Formal lässt sich die mittlere Warenkorbgröße als Quotient aus der Gesamtzahl der Artikel eines Kunden und der Anzahl seiner Bestellungen ausdrücken:

$$\text{Average Basket Size} = \frac{\text{Gesamtzahl Artikel eines Kunden}}{\text{Anzahl seiner Bestellungen}}.$$

Sortimentsmerkmale (*Relative Spending per Product Group*) erfassen die Breite beziehungsweise die Fokussierung der Ausgaben eines Kunden. Sie geben an, welcher Anteil des Gesamtumsatzes auf einzelne Produktgruppen entfällt. Formal kann der Anteil einer Kategorie i als Verhältnis von Umsatz in dieser Kategorie zum Gesamtumsatz des Kunden dargestellt werden:

$$\text{Sortimentsanteil}_i = \frac{\text{Umsatz in Kategorie } i}{\text{Gesamtumsatz des Kunden}}.$$

Ein hoher Wert in einer Kategorie kennzeichnet einen „Spezialisten“, während eine gleichmäßigere Verteilung auf viele Kategorien auf einen „Generalisten“ hinweist.

Eine ausführlichere und beispielhafte Darstellung dieser Konzepte erfolgt in Anhang B.4: Warenkorb- und Sortimentsmerkmale, wo anhand von Illustrationen unterschiedliche Kundentypen wie „Frequent Shopper“, „Bulk Buyer“, „Generalisten“ und „Spezialisten“ gegenübergestellt werden.

4.2 Clustering-Verfahren (K-Means, HDBSCAN - zentrale Eigenschaften)

4.3 Clustertendenz und Modelle Hopkins, K-Means, HDBSCAN

Bevor konkrete Clusterverfahren eingesetzt werden, wird mit der Hopkins-Statistik geprüft, ob die Daten überhaupt eine erkennbare Tendenz zur Bildung von Clustern aufweisen. Werte nahe 0,5 deuten auf eine gleichmäßige Streuung ohne klare Gruppen hin, während Werte oberhalb von 0,75 auf deutliche Clustermuster schließen lassen. Vorgehen, Stichprobenumfang, Reproduzierbarkeit und Ergebniszusammenfassung sind in Anhang C.6: Clustertendenz und Testverfahren dokumentiert.

Aufbauend auf dieser Prüfung werden zwei komplementäre Verfahren eingesetzt, die unterschiedliche Strukturannahmen abdecken. K-Means dient als zentroidbasiertes Referenzverfahren und erzeugt gut interpretierbare Zentroidprofile, ist aber auf annähernd kugelförmige Strukturen ausgerichtet. Exemplarisch wird zunächst ein Startwert von $k = 5$ verwendet, um die Funktionsweise zu demonstrieren. Eine inhaltliche Entscheidung über die tatsächliche Clusterzahl erfolgt hier nicht, sondern systematisch in Kapitel 4: Evaluation und Ergebnisse anhand geeigneter Gütemaße (u. a. Silhouette, Davies-Bouldin) und Stabilitätsprüfungen. Die konkreten Umsetzungsschritte stehen in Anhang C.7: Modellierungsschritte K-Means und HDBSCAN.

Als dichtebasiertes Gegenstück wird HDBSCAN eingesetzt. Das Verfahren erfordert keine feste Clusterzahl, kann lokal unterschiedliche Dichten abbilden und weist dünn besetzte Bereiche als Rauschen aus. Dadurch werden neben größeren Segmenten auch Nischenmuster identifiziert. Parameterwahl und Export der Ergebnisse sind ebenfalls in

Anhang C.7 dokumentiert; die Qualität der resultierenden Lösungen wird in Kapitel 4 anhand dichtegeeigneter Indizes und Stabilitätsmaßen bewertet.

Damit ist die methodische Basis gelegt. Die Hopkins-Statistik begründet die Durchführung einer Clusteranalyse, und die Kombination aus K-Means und HDBSCAN stellt sicher, dass sowohl zentroidbasierte als auch dichte-basierte Strukturen erfasst werden.

4.4 Validierung (Hopkins, Silhouette, DBCV) als Qualitätskontrolle

Damit die mit Clustering identifizierten Muster belastbar sind, müssen ihre Ergebnisse überprüft werden. Dazu dienen Validierungsmaße, die im Folgenden vorgestellt werden.

Die Hopkins-Statistik prüft, ob ein Datensatz überhaupt eine Clustertendenz enthält. Acito (2023), S. 271 beschreiben sie als Maß, „to assess the clustering tendency of a data set by measuring the probability that a uniform data distribution generates a given data set“. Werte über 0,75 deuten auf eine starke Clustertendenz hin, Werte nahe 0,5 auf Zufälligkeit. Bei hochdimensionalen E-Commerce-Daten ist die Aussagekraft jedoch eingeschränkt, weshalb Hopkins in dieser Arbeit nur ergänzend berücksichtigt wird.

Der Silhouette-Koeffizient bewertet die Kohäsion innerhalb von Clustern und die Separation zwischen Clustern. Er kombiniert also zwei Perspektiven: interne Homogenität und externe Abgrenzung. Tsipis und Chorianopoulos (2010), S. 209 nennen die Silhouette als ein zentrales Kriterium der technischen Evaluation. Lai et al. (2025), S. 3063 ergänzen, dass sie ein objektives Maß sei, weil sie „both the cohesion within clusters and the separation between clusters“ berücksichtigt.

Für dichte-basierte Verfahren wie HDBSCAN ist der DBCV-Index geeignet. Moulavi et al. (2014), S. 839 erklären, dass DBCV die „within and between-cluster density connectedness“ misst, während Rauschen explizit berücksichtigt wird. Der Vorteil liegt darin, dass dieses Maß auch für arbiträre Clusterformen robust anwendbar ist (Moulavi et al. (2014), S. 845).

Dabei ist zu berücksichtigen, dass interne Validierungsmaße wie Silhouette oder DBCV lediglich strukturelle Eigenschaften der Clusterlösungen abbilden. Sie garantieren nicht, dass die Segmente geschäftlich relevant sind. Ohne eine anschließende inhaltliche Interpretation besteht die Gefahr, rechnerisch erzeugte Gruppen zu überinterpretieren.

Diese theoretischen Grundlagen bilden den Rahmen, um in der folgenden Untersuchung herauszuarbeiten, welche Segmentstrukturen sich in E-Commerce-Daten verbergen und wie sie so beschrieben werden können, dass sie für Marketing und CRM unmittelbar nutzbar sind.

5 Ergebnisse und Diskussion (CRISP-DM: Modeling und Evaluation - praktisch)

5.1 Preprocessing (Bereinigung, Skalierung, Variablenableitung – Resultate, Details in Anhang C)

Die Datensätze wurden zunächst bereinigt, Duplikate und fehlerhafte Einträge entfernt sowie Rückgaben separat behandelt. Anschließend erfolgte die Vereinheitlichung der Zeitstempel und die Konstruktion abgeleiteter Variablen ($\text{Umsatz} = \text{Preis} \times \text{Menge}$). Über Recency, Frequency und Monetary Value hinaus wurden Merkmale wie Warenkorbgröße und Sortimentsvielfalt gebildet. Zur Vergleichbarkeit aller Variablen wurde eine Standardisierung angewandt. Die detaillierte Umsetzung und die Explorative Datenanalyse sind in Anhang C dokumentiert.

5.2 Clustering (K-Means und HDBSCAN – Ergebnisse und Validierung)

Zur Segmentierung wurden zwei Verfahren angewandt:

K-Means: Für verschiedene Clusterzahlen ($k = 3-8$) berechnet. Die Silhouette-Werte zeigten, dass eine 5-Cluster-Lösung eine gute Balance aus Trennschärfe und Interpretierbarkeit liefert.

HDBSCAN: Automatische Erkennung von Clustern mit variabler Dichte. Neben drei stabilen Segmenten wurde ein Teil der Kunden als Rauschen identifiziert. Der DBCV-Wert bestätigte eine robuste Abgrenzung der Cluster.

Die Validierung verdeutlicht, dass K-Means für kompakte Strukturen geeignet ist, wäh-

rend HDBSCAN flexibler auf unregelmäßige Muster reagiert.

5.3 Interpretation der Segmente (Clusterprofile, Personas, Visualisierungen)

Die Cluster wurden anhand zentraler Kennzahlen interpretiert:

Cluster 1: Häufige Käufer mit mittlerem Warenkorbwert (loyale Stammkunden).

Cluster 2: Seltene Käufer mit hohen Umsätzen (Gelegenheitskunden mit Premium-Fokus).

Cluster 3: Wenige Käufe, geringer Umsatz (Kunden mit Abwanderungsrisiko).

Cluster 4: Sehr hohe Frequenz, kleine Warenkörbe (Schnäppchenjäger).

Cluster 5 (HDBSCAN): Unregelmäßiges Kaufmuster, oft saisonal bedingt (Gelegenheitsnutzer).

Diese Segmente wurden in Personas übersetzt und durch Visualisierungen unterstützt (siehe Anhang D). Daraus lassen sich für Marketing und CRM differenzierte Maßnahmen ableiten, z. B. Reaktivierungskampagnen für Cluster 3 oder exklusive Angebote für Cluster 2.

6 Diskussion

6.1 Vergleich der Verfahren

6.2 Implikationen für Marketing und CRM

6.3 Grenzen und methodische Limitationen

7 Fazit und Ausblick

7.1 Beantwortung der Forschungsfrage

Die Arbeit hat gezeigt, dass sich in E-Commerce-Daten mehrere klar unterscheidbare Segmente identifizieren lassen. Diese unterscheiden sich sowohl in quantitativen Kennzahlen wie Kaufhäufigkeit, Umsatzhöhe und Sortimentspräferenzen als auch in charakteristischen Verhaltensmustern. Mit den aufbereiteten Profilen, Personas und Visualisierungen konnten die Unterschiede verständlich gemacht und in praxisnahe Maßnahmen überführt werden. Damit wurde die Forschungsfrage beantwortet, wie Segmentstrukturen in E-Commerce-Daten sichtbar gemacht und für Marketing und CRM nutzbar aufbereitet werden können.

Die eingesetzten Verfahren haben dabei ihren Zweck erfüllt. K-Means und HDBSCAN dienten als Werkzeuge, um Strukturen sichtbar zu machen. Silhouette und DBCV überprüften die Qualität der Ergebnisse. Die Hopkins-Statistik wurde ergänzend einbezogen, zeigte erwartungsgemäß kaum Clusterfähigkeit und machte deutlich, dass manche Validierungsansätze in diesem Kontext methodisch nur begrenzt aussagekräftig sind.

7.2 Limitationen

Die Ergebnisse basieren auf internen Validitätsmaßen und bieten keinen direkten Nachweis, ob segmentbasierte Maßnahmen in der Praxis zu besseren Erfolgen führen. Auch die Datenbasis weist Einschränkungen auf. Transaktions- und Eventdaten sind häufig unbalanciert, enthalten Ausreißer und bilden nicht alle relevanten Kundenmerkmale ab. Die Generalisierbarkeit auf andere Märkte oder Shops ist daher begrenzt.

7.3 Ausblick (z. B. Text Mining, Echtzeit-Segmentierung)

Für zukünftige Arbeiten empfiehlt sich die Ergänzung um externe Validierung. A/B-Tests und Uplift-Analysen könnten zeigen, ob segmentierte Kampagnen tatsächlich bessere Ergebnisse liefern. Eine Erweiterung der Merkmalsbasis durch Text Mining von Kundenbewertungen oder die Analyse von Session-Daten könnte zusätzliche Differenzierung schaffen. Auch eine dynamische Segmentierung, die sich in Echtzeit an Veränderungen im Verhalten anpasst, wäre ein sinnvoller Schritt. Schließlich sollte die Segmentlogik direkt in CRM-Systeme integriert werden, um einen geschlossenen Kreislauf zwischen Analyse, Umsetzung und Erfolgsmessung zu schaffen.

Was eine [Multi-Faktor-Authentifizierung \(MFA\)](#) ist, wird im Glossar beschrieben.

Literaturverzeichnis

- Acito, F. (2023). *Predictive Analytics with KNIME: Analytics for Citizen Data Scientists*. Springer Nature Switzerland. <https://doi.org/10.1007/978-3-031-45630-5>
- Chakraborty, S., Islam, S. H., und Samanta, D. (2022). *Data Classification and Incremental Clustering in Data Mining and Machine Learning*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-93088-2>
- Datasets - UCI Machine Learning Repository*. (2019, September 21). https://archive.ics.uci.edu/datasets?search=Online+Retail&utm_source=chatgpt.com
- Lai, H., Huang, T., Lu, B., Zhang, S., und Xiaog, R. (2025). Silhouette coefficient-based weighting k-means algorithm. *Neural Computing and Applications*, 37(5), 3061–3075. <https://doi.org/10.1007/s00521-024-10706-0>
- Moulavi, D., Jaskowiak, P. A., Campello, R. J. G. B., Zimek, A., und Sander, J. (2014). Density-Based Clustering Validation. *Proceedings of the 2014 SIAM International Conference on Data Mining*, 839–847. <https://doi.org/10.1137/1.9781611973440.96>
- Tsiptsis, K., und Chorianopoulos, A. (2010). *Data Mining Techniques in CRM: Inside Customer Segmentation* (1. Aufl.). Wiley. <https://doi.org/10.1002/9780470685815>
- Wirth, R., und Hipp, J. (2000). *CRISP-DM: Towards a Standard Process Model for Data Mining*. 11. <http://cs.unibo.it/~danilo.montesi/CBD/Beatriz/10.1.1.198.5133.pdf>
- Zykov, R., Noskov, A., und Anokhin, A. (2022). *Retailrocket recommender system dataset*. Kaggle.com. <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>

Anhang

Hinweis: Alle Abbildungen in den Anhängen tragen das Präfix **A** in ihrer Nummerierung. Das **A** verweist zugleich auf den Anhang und wird fortlaufend gezählt.

Anhangsverzeichnis

Anhang A Modelle und Prozesse	23
A.1 Das CRISP-DM-Modell	24
A.2 Segmentation Management Strategy	24
Anhang B Clustering und RFM	26
B.1 Eindimensionale RFM-Segmentierung	28
B.2 Mehrdimensionale RFM-Segmentierung	28
B.3 Retail-Segmentierung mit RFM	29
Anhang C Data Preparation	47
C.1 Bereinigung (fehlende Werte, Duplikate, Rückgaben)	47
C.2 Skalierung und Transformation	52
C.3 Explorative Datenanalyse (zusätzliche Plots, Tabellen, Code)	56

Anhang A: Prozessmodelle

Dieser Anhang stellt zwei etablierte Prozessmodelle vor, die für Datenanalyse und Segmentierungsstrategien maßgeblich sind. Mit dem CRISP-DM-Referenzmodell wird ein methodischer Rahmen für die systematische Durchführung von Data-Mining-Projekten beschrieben. Ergänzend veranschaulicht die Segmentation Management Strategy, wie analytisch gewonnene Kundensegmente in strategische Marketing- und CRM-Maßnahmen überführt werden können. Die Kombination macht deutlich, dass erfolgreiche Segmentierung sowohl methodisch fundiert als auch organisatorisch eingebettet sein muss.

A.1: Das CRISP-DM-Modell

Die Abbildung A.1 zeigt das CRISP-DM-Referenzmodell mit seinen sechs Phasen Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation und Deployment. Pfeile markieren die zentralen Abhängigkeiten, während der äußere Kreis den iterativen, zyklischen Charakter des Vorgehens unterstreicht. Das Modell ist als allgemeiner Rahmen entworfen und unabhängig von Anwendungsdomäne oder eingesetzter Software.

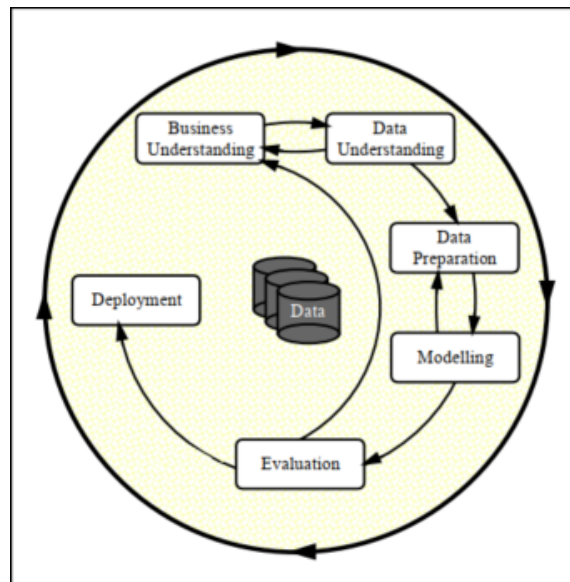


Abbildung A.1: Das CRISP-DM-Modell nach Wirth und Hipp (2000), S. 5

Jede der sechs Phasen ist in spezifische Aufgaben untergliedert, die den Ablauf systematisieren und nachvollziehbar machen. So umfasst Data Understanding das Beschreiben, Erkunden und Überprüfen der Datenqualität, während Data Preparation Aufgaben wie Auswählen, Bereinigen, Konstruieren und Integrieren von Daten vorsieht. Modeling beinhaltet die Wahl geeigneter Verfahren, die Erstellung eines Testdesigns sowie den Aufbau und die Bewertung von Modellen. Evaluation konzentriert sich auf die Überprüfung der Ergebnisse im Hinblick auf die Projektziele, und Deployment behandelt die Bereitstellung und Nutzung der Modelle im Anwendungskontext. Wirth und Hipp betonen außerdem den stark iterativen Charakter des Modells: Ergebnisse einer Phase können jederzeit zu Rücksprüngen führen, wodurch die Arbeitsschritte flexibel angepasst werden können (Wirth und Hipp (2000), S. 6–9).

A.2: Segmentation Management Strategy

Die in Abbildung A.2 dargestellte Roadmap der *Segmentation Management Strategy* zeigt, wie analytisch ermittelte Segmente in eine unternehmerische Nutzung überführt werden können. Sie ergänzt das CRISP-DM-Modell, indem sie die Brücke von der technischen Modellierung hin zur strategischen Implementierung schlägt. Während CRISP-DM beschreibt, wie Daten zu Modellen verarbeitet werden, zeigt die Segmentation Management Strategy, wie diese Modelle in Marketing- und CRM-Maßnahmen eingebettet werden.

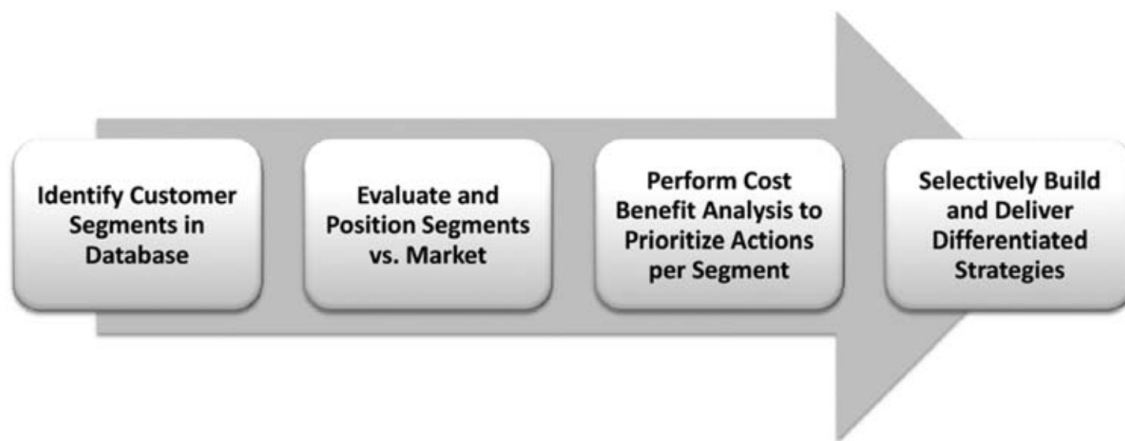


Abbildung A.2: Roadmap der Segmentation Management Strategy nach Tsipitsis und Chorianopoulos (2010), S. 213–216

Die Roadmap besteht aus vier Schritten, die nacheinander zu durchlaufen sind:

Schritt 1: Identify the customer segments in the database

Im ersten Schritt werden die ermittelten Segmente in die Kundendatenbank integriert. Kunden erhalten Scores und werden den Segmenten zugewiesen. Tsipitsis und Chorianopoulos (2010), S. 213 betonen: *“Initially the segmentation results should be deployed in the customer database. Customers should be scored and assigned into segments.”* Damit ist die organisatorische Grundlage geschaffen, auf der weitere Analysen und Maßnahmen aufbauen.

Schritt 2: Evaluate and position the segments

In einem zweiten Schritt werden die Segmente bewertet und im Marktumfeld positioniert. Dazu gehören Marktumfragen und qualitative Analysen, die Segmente mit zusätzlichen Informationen wie Bedürfnissen, Lebensstil oder Status anreichern. Zudem sollen Wettbe-

werbsvorteile, Chancen und Risiken identifiziert werden. Die Autoren halten fest, dass *“the possible opportunities and threats should be investigated in order to select the key segments to be targeted with marketing activities tailored to their profiles and needs”* (Tsiptsis und Chorianopoulos (2010), S. 214).

Schritt 3: Perform cost-benefit analysis to prioritize actions per segment

Darauf folgt eine Kosten-Nutzen-Analyse, die die Profitabilität einzelner Segmente in Relation zu den erwarteten Kosten setzt. Behavioral Drivers wie Kaufhäufigkeit oder Abwanderungsrisiken spielen dabei eine zentrale Rolle. Ziel ist die Priorisierung der Segmente nach ökonomischer Relevanz. *“Effective segmentation management requires insight into the factors that drive customer behaviors ... Finally, a cost-benefit analysis can enable the prioritization of the marketing actions to be implemented”* (Tsiptsis und Chorianopoulos (2010), S. 215).

Schritt 4: Build and deliver differentiated strategies

Im vierten Schritt werden die Segmente mit maßgeschneiderten Strategien adressiert. Spezialisierte Teams entwickeln und implementieren differenzierte Kampagnen, CRM-Maßnahmen und Produktstrategien. Dies umfasst Aspekte wie Kanalmanagement, Marketingservices, Produktmanagement und Markenführung. Tsiptsis und Chorianopoulos (2010), S. 216 schreiben hierzu: *“Each team should build and deliver specialized marketing strategies to improve customer handling, develop the relationship with customers, and increase the profitability of each segment.”*

Die Roadmap verdeutlicht, dass Segmentierung mehr ist als ein analytischer Prozessschritt. Sie muss in eine organisatorische und strategische Logik eingebettet werden, damit Segmente nicht nur statistisch existieren, sondern im Unternehmen handlungsleitend wirken. CRISP-DM liefert dafür den methodischen Rahmen, die Segmentation Management Strategy ergänzt ihn um eine explizite Managementperspektive.

Anhang B: Clustering und RFM

Dieser Anhang erläutert die RFM-Segmentierung als ein klassisches Instrument der Kundenanalyse. Im Mittelpunkt stehen die Dimensionen Recency, Frequency und Monetary Value, mit denen Kundenverhalten in strukturierte Kategorien überführt werden kann. Die Darstellung zeigt sowohl einfache eindimensionale Ansätze als auch mehrdimensionale Ausprägungen, die eine differenziertere Sicht auf Kaufmuster und Kundenwert ermöglichen. Damit wird nachvollziehbar, weshalb RFM-Modelle eine wichtige Grundlage für moderne Verfahren der Kundensegmentierung bilden.

B.1: Eindimensionale RFM-Segmentierung

Eindimensionale RFM-Segmentierung betrachtet jeweils nur eine Dimension des RFM-Modells – also Recency, Frequency oder Monetary – isoliert.

Dabei werden Kunden typischerweise in Quantile eingeteilt. So werden beispielsweise die 20 % der Kunden mit der höchsten Kaufhäufigkeit in die oberste Kategorie ($F = 5$) eingeordnet, während die 20 % mit der geringsten Häufigkeit die niedrigste Stufe ($F = 1$) bilden (Tsiptsis und Chorianopoulos, 2010, S. 333–336).

Dieser Ansatz ist leicht verständlich und ermöglicht eine schnelle Einordnung von Kunden nach einem einzelnen Aspekt des Kaufverhaltens. Allerdings bildet er die Gesamtkomplexität der Kundenbeziehungen nur eingeschränkt ab, da Interaktionen zwischen den Dimensionen unberücksichtigt bleiben.

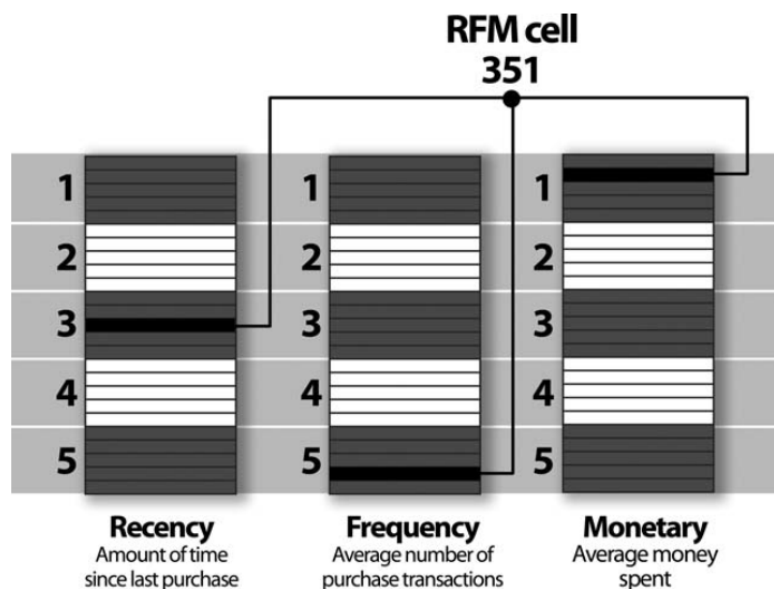


Abbildung A.3: Beispiel für eindimensionale RFM-Einteilungen in fünf Quantile je Dimension. Hervorgehoben ist die RFM-Zelle 351 (Recency = 3, Frequency = 5, Monetary = 1) nach Tsiptsis und Chorianopoulos (2010), S. 336.

B.2: Mehrdimensionale RFM-Segmentierung

Die RFM-Analyse kombiniert drei Dimensionen: Recency, Frequency und Monetary Value. Tsiptsis und Chorianopoulos (2010), S. 336–345 beschreiben RFM als bewährten Ansatz, um

Kundenwerte differenziert abzubilden. Neben der klassischen Zellenbildung (z. B. Quintile) kann RFM auch als Input für Clustering genutzt werden. Dadurch entstehen mehrdimensionale Segmente, die sich deutlich voneinander unterscheiden. Erweiterungen des Modells beziehen zusätzliche Merkmale wie Warenkorbgrößen oder Sortimentspräferenzen ein, um die Aussagekraft weiter zu erhöhen.

B.3: Retail-Segmentierung mit RFM

Ergänzend zur theoretischen Einführung im Hauptteil wird hier die Zellenlogik von RFM veranschaulicht.

Die folgende programmgenerierte Darstellung zeigt alle möglichen Kombinationen von *Recency* und *Frequency* auf einer Ebene mit festem *Monetary*-Wert. Die Zelle 351 ist dabei als Beispiel hervorgehoben.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')

# RFM-Dimensionen
recency_range = range(1, 6)      # 1-5
frequency_range = range(1, 6)    # 1-5
monetary_level = 1                # Feste Ebene bei Z=1

# Vollständige Ebene bei Monetary = 1
for r in recency_range:
    for f in frequency_range:
        code = f"{r}-{f}-{monetary_level}"

        # Hervorhebung für Code "351"
        if code == "351":
            color = "red"          # Besondere Hervorhebung in rot
            text_color = "white"   # Weiße Schrift auf rot
```

```

else:
    color = "lightgrey"
    text_color = "black"      # Schwarze Schrift auf grau

# Korrigierte Positionierung: x=r, y=f, z=monetary_level
ax.bar3d(
    x=r-0.5, y=f-0.5, z=monetary_level-0.1, # Startposition (Ecke des
Würfels)
    dx=1, dy=1, dz=0.2, # Größe
    color=color, edgecolor='black', alpha=0.9 # Weniger transparent
)

ax.text(
    r, f, monetary_level + 0.05, # Leicht über der Oberfläche
    code,
    color=text_color,
    ha='center', va='center',
    fontsize=14, # Noch größere Schrift
    weight='bold', # ALLE Texte fett
    zorder=100 # Text immer im Vordergrund
)

# Achsentitel (korrekte Zuordnung)
ax.set_xlabel("Recency", fontsize=12, weight='bold')
ax.set_ylabel("Frequency", fontsize=12, weight='bold')
ax.set_zlabel("Monetary", fontsize=12, weight='bold')

# Achsenticks
ax.set_xticks(range(1, 6))
ax.set_xticklabels(['1', '2', '3', '4', '5'])
ax.set_yticks(range(1, 6))
ax.set_yticklabels(['1', '2', '3', '4', '5'])
ax.set_zticks(range(1, 6))

```

```
# Achsgrenzen  
ax.set_xlim(0.5, 5.5)
```

```
ax.set_ylim(0.5, 5.5)
```

```
ax.set_zlim(0.5, 5.5)
```

```
# Seitenverhältnis & Perspektive  
ax.set_box_aspect([1, 1, 1]) # Gleiche Proportionen  
ax.view_init(elev=25, azim=45) # Bessere Sicht  
ax.grid(True)  
  
# Überschrift mit RFM-Score Berechnung  
ax.set_title("RFM-Würfel: Kontinuierlicher Score-Ansatz\n" +  
             "RFM Score = (R×wR) + (F×wF) + (M×wM)\n" +  
             "Ebene bei Monetary = 1 | Code 351 hervorgehoben",  
             fontsize=13, weight='bold', pad=20)  
  
# Layout-Anpassung ohne Warnung  
plt.subplots_adjust(left=0.1, right=0.9, top=0.85, bottom=0.1) # Mehr Platz  
für Titel  
plt.show()
```

RFM-Würfel: Kontinuierlicher Score-Ansatz
RFM Score = (R×wR) + (F×wF) + (M×wM)
Ebene bei Monetary = 1 | Code 351 hervorgehoben

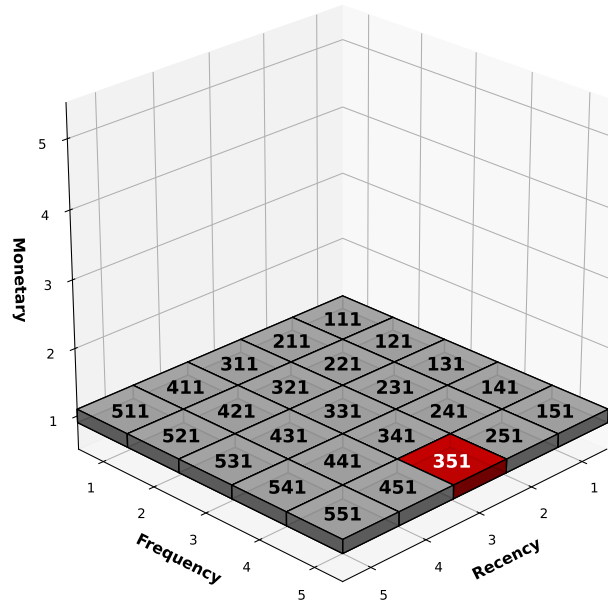


Abbildung A.4: Eigene Darstellung eines RFM-Würfels

Die Abbildung zeigt einen zweidimensionalen Ausschnitt des RFM-Modells in Würfelform:

- Die x-Achse steht für *Recency* (Zeit seit dem letzten Kauf).
- Die y-Achse repräsentiert *Frequency* (Kaufhäufigkeit).
- Die z-Achse bildet *Monetary* (Umsatzhöhe) ab. In dieser Darstellung ist die Ebene mit *Monetary* = 1 ausgewählt.

Jede Zelle im Würfel steht für eine Kombination dieser drei Dimensionen und wird durch einen dreistelligen Code gekennzeichnet (z. B. 351 = Recency 3, Frequency 5, Monetary 1).

Die hervorgehobene Zelle 351 veranschaulicht exemplarisch, wie einzelne Kundengruppen identifiziert werden können. Damit wird deutlich, dass das RFM-Schema ein Raster bildet, das eine differenzierte Beschreibung von Kundenverhalten ermöglicht.

B.4: Warenkorb- und Sortimentsmerkmale

Dieser Abschnitt illustriert die im Hauptteil beschriebenen Konzepte *Warenkorbgrößen* und *Sortimentsmerkmale* anhand von Beispielen und Visualisierungen. Während Kapitel 4.1 die theoretischen Grundlagen und formalen Definitionen eingeführt hat, werden hier exemplarische Kundentypen wie „Frequent Shopper“, „Bulk Buyer“, „Generalisten“ und „Spezialisten“ dargestellt. Auf diese Weise wird deutlich, wie sich die abstrakten Kennzahlen in der Praxis auf konkrete Verhaltensmuster übertragen lassen.

Warenkorbgrößen

Die Abbildung A.5 zeigt zwei kontrastierende Kundentypen:

- Kunde A („Frequent Shopper“) tätigt häufige, kleine Einkäufe.
- Kunde B („Bulk Buyer“) kauft selten, aber in großen Mengen.

Obwohl beide die gleiche Gesamtzahl an Artikeln kaufen, unterscheiden sie sich im Bestellverhalten fundamental. Diese Unterschiede lassen sich durch die *Average Basket Size* quantifizieren.

```
import matplotlib.pyplot as plt
import matplotlib.path_effects as path_effects
import numpy as np

# Daten für beide Kundentypen
customers = ["Kunde A\n(Frequent Shopper)", "Kunde B\n(Bulk Buyer)"]
basket_size = [2, 20]
order_count = [10, 2]
total_items = [20, 40]

# 3D-Farbpalette
colors_primary = ['#FF6B6B', '#4ECDC4']
colors_secondary = ['#E74C3C', '#45B7D1']
colors_scatter = ['#FF1744', '#00BCD4']
```



```

fig = plt.figure(figsize=(16, 10))

# Layout mit GridSpec - ERWEITERT für Kennzahlen
gs = fig.add_gridspec(3, 3, height_ratios=[2.5, 1, 0.8], width_ratios=[1, 1,
0.8],
                        hspace=0.4, wspace=0.3, top=0.88, bottom=0.08)

# 3D Bar Charts
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1])
ax3 = fig.add_subplot(gs[1, 0])
ax4 = fig.add_subplot(gs[:2, 2]) # Rechts über 2 Zeilen
ax5 = fig.add_subplot(gs[2, :2]) # NEUE Kennzahlen-Zeile

# 1. Warenkorbgröße mit besserer Y-Achsen-Skalierung
bars1 = ax1.bar(customers, basket_size, color=colors_primary, alpha=0.9,
                edgecolor='black', linewidth=2,
                width=0.6)
ax1.set_title('Durchschnittliche Warenkorbgröße', fontweight='bold',
             fontsize=12, pad=15)
ax1.set_ylabel('Artikel pro Bestellung', fontweight='bold')
ax1.grid(True, alpha=0.3, linestyle='--')
ax1.set_facecolor('#F8F9FA')
ax1.set_ylim(0, max(basket_size) * 1.3)

```

Terminal-Output

(0.0, 26.0)

```

# Werte besser positioniert
for i, (bar, value) in enumerate(zip(bars1, basket_size)):
    text = ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
max(basket_size) * 0.05,

```

```

        str(value), ha='center', va='bottom', fontweight='bold',
fontsize=14,
        color='white')
    text.set_path_effects([path_effects.withStroke(linewidth=2,
foreground='black')])

# 2. Anzahl Bestellungen mit besserer Y-Achsen-Skalierung
bars2 = ax2.bar(customers, order_count, color=colors_secondary, alpha=0.9,
                edgecolor='black', linewidth=2,
                width=0.6)
ax2.set_title('Anzahl Bestellungen', fontweight='bold', fontsize=12, pad=15)
ax2.set_ylabel('Bestellungen gesamt', fontweight='bold')
ax2.grid(True, alpha=0.3, linestyle='--')
ax2.set_facecolor('#F8F9FA')
ax2.set_ylim(0, max(order_count) * 1.3)

```

Terminal-Output

(0.0, 13.0)

```

# Werte besser positioniert
for i, (bar, value) in enumerate(zip(bars2, order_count)):
    text = ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
max(order_count) * 0.05,
                    str(value), ha='center', va='bottom', fontweight='bold',
fontsize=14,
                    color='white')
    text.set_path_effects([path_effects.withStroke(linewidth=2,
foreground='black')])

# 3. Scatter Plot GRÖßER mit besserer Achsen-Skalierung
ax3.scatter(order_count, basket_size, s=[500, 500],
            c=colors_scatter, alpha=0.8,
            edgecolors='black', linewidth=2,

```

```

        zorder=5)
ax3.set_xlabel('Anzahl Bestellungen', fontweight='bold', fontsize=11)
ax3.set_ylabel('Warenkorbgröße', fontweight='bold', fontsize=11)
ax3.set_title('Kundentypen im Vergleich', fontweight='bold', fontsize=13,
pad=15)
ax3.grid(True, alpha=0.3, linestyle='--')
ax3.set_facecolor('#F8F9FA')

# Bessere Achsen-Grenzen für mehr Platz
ax3.set_xlim(0, max(order_count) * 1.2)

```

Terminal-Output

(0.0, 12.0)

```
ax3.set_ylim(0, max(basket_size) * 1.2)
```

Terminal-Output

(0.0, 24.0)

```

# Annotationen mit 3D-Effekt - größer
for i, customer in enumerate(['A', 'B']):
    ax3.annotate(f'Kunde {customer}',
                xy=(order_count[i], basket_size[i]),
                xytext=(10, 10), textcoords='offset points',
                fontweight='bold', fontsize=12, color='white',
                bbox=dict(boxstyle="round,pad=0.4",
                        facecolor=colors_scatter[i], alpha=0.8,
                        edgecolor='black', linewidth=2))

# 4. Strategische Implikationen mit kompakterem Layout
ax4.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```
# 3D-artiger Hintergrund
ax4.add_patch(plt.Rectangle((0, 0), 1, 1, transform=ax4.transAxes,
                           facecolor='lightgray', alpha=0.2,
                           edgecolor='black', linewidth=1))

# Titel mit 3D-Box
ax4.text(0.05, 0.95, 'Strategische Implikationen', fontsize=12,
         fontweight='bold',
         transform=ax4.transAxes, color='white',
         bbox=dict(boxstyle="round,pad=0.3", facecolor='steelblue', alpha=0.8,
         edgecolor='black'))

# Kunde A Implikationen - kompakter
ax4.text(0.05, 0.85, 'Kunde A (Frequent Shopper):', fontsize=10,
         fontweight='bold',
         transform=ax4.transAxes, color='white',
         bbox=dict(boxstyle="round,pad=0.2", facecolor='#FF6B6B', alpha=0.8,
         edgecolor='black'))

implications_a = ['• Häufige Kontaktpunkte', '• Loyalitätsprogramme', '•
Cross-Selling-Potenzial']
for i, impl in enumerate(implications_a):
    ax4.text(0.05, 0.79 - i*0.05, impl, fontsize=9, transform=ax4.transAxes,
             bbox=dict(boxstyle="round,pad=0.15", facecolor='white', alpha=0.8,
             edgecolor='gray'))

# Kunde B Implikationen - kompakter
ax4.text(0.05, 0.60, 'Kunde B (Bulk Buyer):', fontsize=10, fontweight='bold',
         transform=ax4.transAxes, color='white',
         bbox=dict(boxstyle="round,pad=0.2", facecolor='#4ECDC4', alpha=0.8,
         edgecolor='black'))
```

```

implications_b = ['• Planungsorientiert', '• Volumenrabatte', '•
Lagerservice']
for i, impl in enumerate(implications_b):
    ax4.text(0.05, 0.54 - i*0.05, impl, fontsize=9, transform=ax4.transAxes,
             bbox=dict(boxstyle="round,pad=0.15", facecolor='white', alpha=0.8,
edgecolor='gray'))

# Fazit mit 3D-Effekt - kompakter
ax4.text(0.05, 0.35, 'Fazit:', fontsize=10, fontweight='bold',
         transform=ax4.transAxes, color='white',
         bbox=dict(boxstyle="round,pad=0.2", facecolor='darkgreen', alpha=0.8,
edgecolor='black'))

ax4.text(0.05, 0.25, 'Beide Typen wertvoll, aber\ndunterschiedliche Ansprache
nötig',
         fontsize=9, transform=ax4.transAxes, style='italic',
         bbox=dict(boxstyle="round,pad=0.15", facecolor='lightyellow',
alpha=0.8, edgecolor='orange'))

# NEUE Kennzahlen-Ausgabe für Warenkorbanalyse
ax5.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

kennzahlen_text = f"""WARENKORBANALYSE: Kunde A - Warenkorbgröße:
{basket_size[0]} Artikel | Bestellungen: {order_count[0]}/Periode | Kunde B -
Warenkorbgröße: {basket_size[1]} Artikel | Bestellungen:
{order_count[1]}/Periode | Verhältnis:
{basket_size[1]/basket_size[0]:.1f}:1"""

ax5.text(0.5, 0.5, kennzahlen_text, fontsize=10,

```

```

transform=ax5.transAxes, ha='center', va='center',
fontfamily='monospace', fontweight='bold',
bbox=dict(boxstyle="round,pad=0.3", facecolor='gold',
          alpha=0.8, edgecolor='black', linewidth=2))

# 3D-Titel höher positioniert
plt.suptitle('Warenkorbstrategien: 3D-Analyse verschiedener Kundentypen',
            fontsize=16, fontweight='bold', y=0.96,
            bbox=dict(boxstyle="round,pad=0.5", facecolor='gold', alpha=0.8,
                    edgecolor='black', linewidth=2))

plt.tight_layout()
plt.show()

```

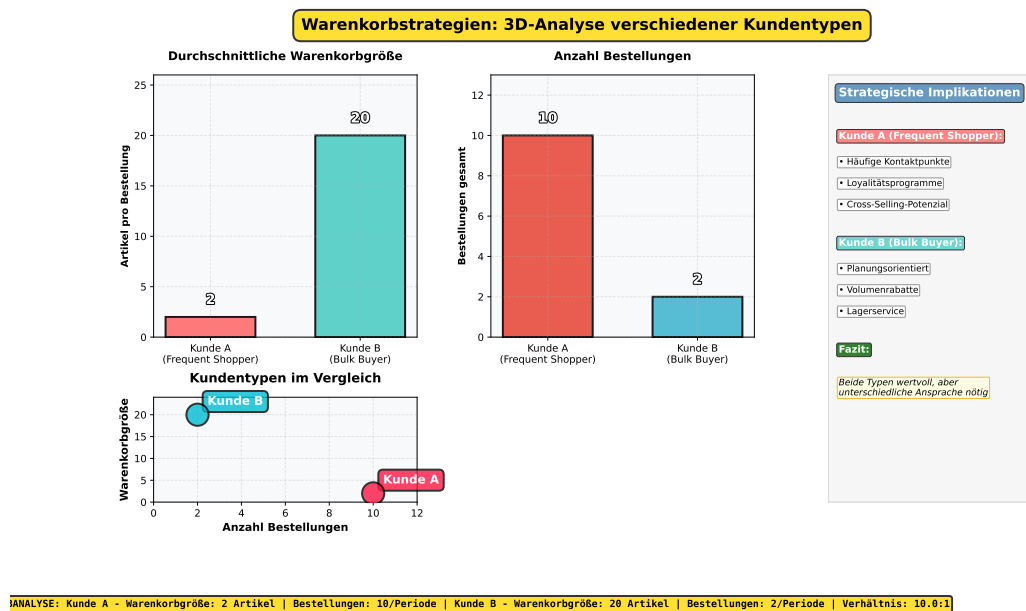


Abbildung A.5: Warenkorbanalyse verschiedener Kundentypen mit Durchschnittswerten, Bestellhäufigkeit und Vergleichsdarstellung

Die Visualisierung verdeutlicht, dass beide Kundentypen für ein Unternehmen wertvoll sein können, jedoch unterschiedliche Marketingstrategien erfordern. Während

Frequent Shopper auf Loyalitätsprogramme und Cross-Selling reagieren, benötigen Bulk Buyer volumenorientierte Anreize und Serviceleistungen.

Sortimentsmerkmale

Die Abbildung A.6 stellt das Verhalten eines Generalisten (Kunde C) und eines Spezialisten (Kunde D) gegenüber.

```
import matplotlib.pyplot as plt
import matplotlib.path_effects as path_effects
import numpy as np

# Beispiel: Sortimentsanteile
categories = ["Elektronik", "Kleidung", "Bücher", "Sport", "Haushalt"]

# Generalist: gleichmäßige Verteilung
generalist = [20, 20, 20, 20, 20]

# Spezialist: starke Fokussierung auf eine Kategorie
specialist = [80, 8, 5, 4, 3]

# Intensivere 3D-Farbpalette mit Verläufen
colors_generalist = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A', '#98D8C8']
colors_specialist = ['#E74C3C', '#3498DB', '#F39C12', '#9B59B6', '#2ECC71']

fig = plt.figure(figsize=(16, 9))

# Layout mit GridSpec - MEHR PLATZ OBEN für Titel
gs = fig.add_gridspec(3, 3, height_ratios=[2.5, 1, 0.8], width_ratios=[1, 1, 0.8],
                      hspace=0.5, wspace=0.3, top=0.82, bottom=0.08) # top reduziert für mehr Platz

# Charts definieren - ALLE WEITER UNTEN
ax1 = fig.add_subplot(gs[0, 0]) # Pie Chart 1
```

```

ax2 = fig.add_subplot(gs[0, 1]) # Pie Chart 2
ax3 = fig.add_subplot(gs[:2, 2]) # Rechts über 2 Zeilen
ax4 = fig.add_subplot(gs[1, :2]) # Tabelle
ax5 = fig.add_subplot(gs[2, :2]) # Kennzahlen-Zeile (bleibt)

# Generalist Pie Chart mit 3D-Effekt
wedges1, texts1, autotexts1 = ax1.pie(generalist, labels=categories,
autopct='%1.0f%%',

                                colors=colors_generalist,
                                explode=(0.08, 0.08, 0.08, 0.08, 0.08),
                                shadow=False, # Schatten entfernt
                                startangle=45,
                                wedgeprops={'edgecolor': 'black',
                                              'linewidth': 2,
                                              'linestyle': '-',
                                              'alpha': 0.9},
                                textprops={'fontsize': 9,
                                              'fontweight': 'bold',
                                              'color': 'white'})

ax1.set_title("Kunde C: Generalist\n(Gleichmäßige Diversifikation)",
              fontsize=12, fontweight='bold', pad=15)

# Spezialist Pie Chart mit 3D-Effekt
wedges2, texts2, autotexts2 = ax2.pie(specialist, labels=categories,
autopct='%1.0f%%',

                                colors=colors_specialist,
                                explode=(0.15, 0.03, 0.03, 0.03, 0.03),
                                shadow=False, # Schatten entfernt
                                startangle=45,
                                wedgeprops={'edgecolor': 'black',
                                              'linewidth': 3,
                                              'linestyle': '-',
                                              'alpha': 0.9},

```



```

        textprops={'fontsize': 9,
                    'fontweight': 'bold',
                    'color': 'white'})

ax2.set_title("Kunde D: Spezialist\n(Fokus auf Elektronik)",
              fontsize=12, fontweight='bold', pad=15)

# Extra weiße, fette Schrift für 3D-Effekt
for autotext in autotexts1:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
    autotext.set_fontsize(11)
    autotext.set_path_effects([path_effects.withStroke(linewidth=3,
foreground='black')])

for autotext in autotexts2:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
    autotext.set_fontsize(11)
    autotext.set_path_effects([path_effects.withStroke(linewidth=3,
foreground='black')])

# Analyse-Panel rechts mit 3D-Box
ax3.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

# 3D-artiger Hintergrund
ax3.add_patch(plt.Rectangle((0, 0), 1, 1, transform=ax3.transAxes,
                             facecolor='lightgray', alpha=0.3,
                             edgecolor='black', linewidth=2))

```

```

ax3.text(0.05, 0.95, 'Sortiments-Analyse', fontsize=14, fontweight='bold',
        transform=ax3.transAxes,
        bbox=dict(boxstyle="round,pad=0.3", facecolor='steelblue', alpha=0.7,
edgecolor='black'))

# Kunde C Statistiken mit 3D-Boxen
ax3.text(0.05, 0.85, 'Kunde C (Generalist):', fontsize=11, fontweight='bold',
        transform=ax3.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='#FF6B6B', alpha=0.8,
edgecolor='black'))

stats_c = ['• 5 aktive Kategorien', '• Diversitäts-Index: 1.00',
          '• Gleichmäßige Verteilung', '• Cross-Selling-Potenzial']
for i, stat in enumerate(stats_c):
    ax3.text(0.05, 0.78 - i*0.04, stat, fontsize=10, transform=ax3.transAxes,
            bbox=dict(boxstyle="round,pad=0.2", facecolor='white', alpha=0.8,
edgecolor='gray'))

# Kunde D Statistiken mit 3D-Boxen
ax3.text(0.05, 0.55, 'Kunde D (Spezialist):', fontsize=11, fontweight='bold',
        transform=ax3.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='#E74C3C', alpha=0.8,
edgecolor='black'))

stats_d = ['• 5 Kategorien, 1 dominant', '• Diversitäts-Index: 0.37',
          '• 80% Fokus auf Elektronik', '• Category-Management']
for i, stat in enumerate(stats_d):
    ax3.text(0.05, 0.48 - i*0.04, stat, fontsize=10, transform=ax3.transAxes,
            bbox=dict(boxstyle="round,pad=0.2", facecolor='white', alpha=0.8,
edgecolor='gray'))

# Strategische Implikationen - VERKÜRZT
ax3.text(0.05, 0.25, 'Strategische Implikationen:', fontsize=11,
fontweight='bold',

```

```

        transform=ax3.transAxes, color='white',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='darkgreen', alpha=0.8,
edgecolor='black'))

ax3.text(0.05, 0.18, 'Verschiedene Strategien erfordern\ningepasste Ansätze',
fontSize=10,
        transform=ax3.transAxes, style='italic',
        bbox=dict(boxstyle="round,pad=0.2", facecolor='lightyellow',
alpha=0.8, edgecolor='orange'))

# 3D-Tabelle unten
ax4.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

# Tabellen-Daten
table_data = [
    ['Merkmal', 'Generalist (Kunde C)', 'Spezialist (Kunde D)'],
    ['Kategorien aktiv', '5 (alle gleichmäßig)', '5 (1 dominant)'],
    ['Konzentration', '20% pro Kategorie', '80% Elektronik'],
    ['Diversität', 'Hoch (1.00)', 'Niedrig (0.37)'],
    ['Marketing-Fokus', 'Cross-Selling', 'Category-Expertise']
]

# Tabelle mit 3D-Effekt
table = ax4.table(cellText=table_data[1:],
                  colLabels=table_data[0],
                  cellLoc='center',
                  loc='center',
                  bbox=[0, 0, 1, 1])

table.auto_set_font_size(False)

```

```

table.set_fontsize(9)
table.scale(1, 2.2)

# 3D-Header-Stil
for i in range(3):
    table[(0, i)].set_facecolor('#2C3E50')
    table[(0, i)].set_text_props(weight='bold', color='white')
    table[(0, i)].set_edgecolor('black')
    table[(0, i)].set_linewidth(2)

# 3D-Zeilen mit Farbverläufen
colors_rows = ['#ECF0F1', '#BDC3C7']
for i in range(1, 5):
    for j in range(3):
        table[(i, j)].set_facecolor(colors_rows[i % 2])
        table[(i, j)].set_edgecolor('black')
        table[(i, j)].set_linewidth(1)

# Kennzahlen-Ausgabe (BLEIBT WO SIE IST)
ax5.axis('off')

```

Terminal-Output

```
(np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

```

kennzahlen_text = f""""SORTIMENTSANALYSE: Kunde C - Konzentration:
{max(generalist)}% (niedrig) | Kunde D - Konzentration: {max(specialist)}%
(sehr hoch) | Verhältnis Hauptkategorie:
{max(specialist)/max(generalist):.1f}:1""""

ax5.text(0.5, 0.5, kennzahlen_text, fontsize=10,
         transform=ax5.transAxes, ha='center', va='center',
         fontfamily='monospace', fontweight='bold',
         bbox=dict(boxstyle="round,pad=0.3", facecolor='gold',

```

```

alpha=0.8, edgecolor='black', linewidth=2))

# Titel HÖHER positioniert mit mehr Abstand
plt.suptitle('Sortimentsverhalten: Generalist vs. Spezialist',
             fontsize=18, fontweight='bold', y=0.98, # Höher positioniert
             bbox=dict(boxstyle="round,pad=0.5", facecolor='gold', alpha=0.8,
             edgecolor='black', linewidth=3))

plt.tight_layout()
plt.show()

```

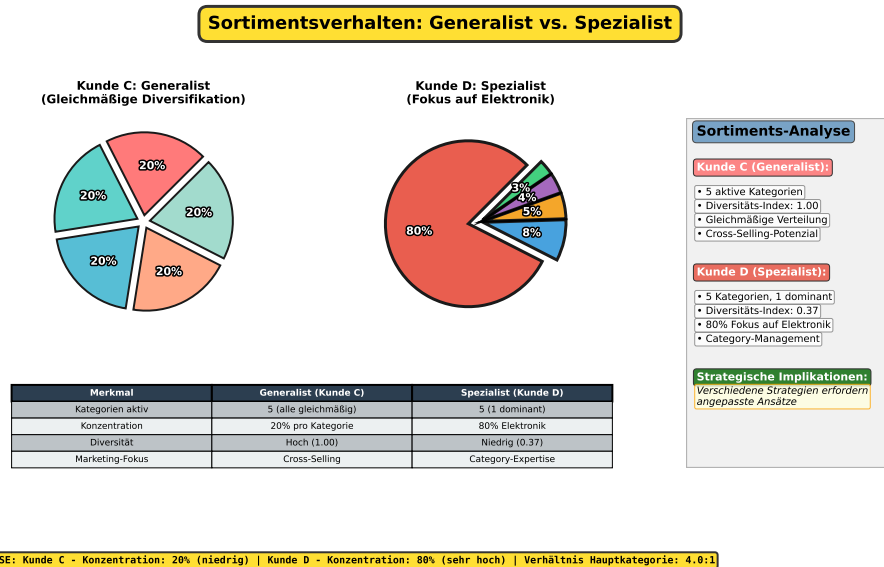


Abbildung A.6: Sortimentsverhalten: Vergleich von Generalisten (gleichmäßige Diversifikation) und Spezialisten (dominante Kategorie)

Kunde C verteilt seine Ausgaben gleichmäßig über verschiedene Kategorien, was sich in einem hohen Diversitätsindex widerspiegelt und Cross-Selling-Potenzial bietet. Kunde D hingegen fokussiert sich stark auf eine einzige Kategorie, was eine spezialisierte Ansprache im Category-Management erforderlich macht.

Anhang C: Datenaufbereitung und Voranalysen

Dieser Anhang dokumentiert die technischen Schritte, die den Haupttext flankieren. Die Abschnitte C.1 bis C.5 stützen die in Abschnitt 3.2 beschriebenen Datensätze sowie die praktische Aufbereitung in Abschnitt 5.1. Abschnitt C.6 liefert die Vorprüfung der Clustertendenz und bildet damit die Brücke zwischen Abschnitt 4.3 Validierung theoretisch und Abschnitt 5.2 Clustering Ergebnisse. Abschnitt C.7 enthält die Implementierung der in Abschnitt 4.2 beschriebenen Verfahren und erzeugt die in Abschnitt 5.2 ausgewerteten Clusterzuordnungen. Die inhaltliche Deutung der Segmente erfolgt in Abschnitt 5.3.

Der Anhang ist wie folgt gegliedert:

1. Variablenbeschreibung und Quellen (C.1)
2. Abbildung auf ein einheitliches Ereignisschema (C.2)
3. Bereinigungsschritte und Qualitätsregeln (C.3)
4. Normalisierung der Zeitstempel und Kalendermerkmale (C.4)
5. Zusammenfassung des bereinigten Bestands und Plausibilisierung (C.5)
6. Clustertendenz mit der Hopkins-Statistik als Vorprüfung
7. Modellierungsschritte in Python für K-Means und HDBSCAN

In den Abschnitten C.1–C.5 wird mit R gearbeitet, um die Daten zu laden, bereinigen und in ein konsistentes Schema zu überführen. Ab C.6 kommen Python und spezialisierte Bibliotheken für Clustertendenz, Modellierung und Evaluation zum Einsatz. Diese Aufgabenteilung folgt der Praxis vieler Forschungsarbeiten: R für transparente Datenaufbereitung, Python für Machine Learning.

C.1 Variablenbeschreibung und Quellen

```
## eval=TRUE
## echo=TRUE
## include=TRUE
## results=TRUE
## message=FALSE
## warning=FALSE

# Pakete laden
library(readr)
library(dplyr)
library(here)

# Versionen dokumentieren
pkgs <- c("readr", "dplyr", "here")
vers <- sapply(pkgs, function(p) as.character(utils::packageVersion(p)))
attached <- pkgs %in% sub("^package:", "", search())
data.frame(package = pkgs, version = vers, attached = attached)
```

Terminal-Output

	package	version	attached
readr	readr	2.1.5	TRUE
dplyr	dplyr	1.1.4	TRUE
here	here	1.0.1	TRUE

```
# Pfade bauen
p_events <- here("data", "RR", "events.csv")
p_item1 <- here("data", "RR", "item_properties_part1.csv")
p_item2 <- here("data", "RR", "item_properties_part2.csv")
p_cat <- here("data", "RR", "category_tree.csv")
p_or2 <- if (file.exists(here("data", "OR2", "online_retail_II.csv"))) {
  here("data", "OR2", "online_retail_II.csv")
}
```

```

    } else {
      here("data", "OR2", "online_retail_ll.csv")
    }

# Laden
events      <- read_csv(p_events, show_col_types = FALSE)
item_props_1 <- read_csv(p_item1,  show_col_types = FALSE)
item_props_2 <- read_csv(p_item2,  show_col_types = FALSE)
category_tree <- read_csv(p_cat,   show_col_types = FALSE)
online_retail <- read_csv(p_or2,   show_col_types = FALSE)

# Erste 5 Zeilen je Datensatz
head(events, 5)

```

 Terminal-Output

```

# A tibble: 5 x 5
  timestamp visitorid event itemid transactionid
  <dbl>      <dbl> <chr>  <dbl>      <dbl>
1 1433221332117 257597 view  355908      NA
2 1433224214164 992329 view  248676      NA
3 1433221999827 111016 view  318965      NA
4 1433221955914 483717 view  253185      NA
5 1433221337106 951259 view  367447      NA

```

```
head(item_props_1, 5)
```

 Terminal-Output

```

# A tibble: 5 x 4
  timestamp itemid property value
  <dbl>    <dbl> <chr>    <chr>
1 1435460400000 460429 categoryid 1338
2 1441508400000 206783 888      1116713 960601 n277.200
3 1439089200000 395014 400      n552.000 639502 n720.000 424566
4 1431226800000 59481 790      n15360.000
5 1431831600000 156781 917      828513

```



```
head(item_props_2, 5)
```

Terminal-Output

```
# A tibble: 5 x 4
  timestamp itemid property value
  <dbl>    <dbl> <chr>    <chr>
1 1433041200000 183478 561      769062
2 1439694000000 132256 976      n26.400 1135780
3 1435460400000 420307 921      1149317 1257525
4 1431831600000 403324 917      1204143
5 1435460400000 230701 521      769062
```

```
head(category_tree, 5)
```

Terminal-Output

```
# A tibble: 5 x 2
  categoryid parentid
  <dbl>    <dbl>
1    1016      213
2     809      169
3     570       9
4    1691     885
5     536    1691
```

```
head(online_retail, 5)
```

Terminal-Output

```
# A tibble: 5 x 8
  Invoice StockCode Description Quantity InvoiceDate Price `Customer ID
  <chr>   <chr>    <chr>         <dbl> <dtm>    <dbl>      <dbl>
1 489434  85048    "15CM CHRI~         12 2009-12-
01 07:45:00 6.95      13085
2 489434  79323P    "PINK CHER~         12 2009-12-
01 07:45:00 6.75      13085
3 489434  79323W    "WHITE CHE~         12 2009-12-
01 07:45:00 6.75      13085
4 489434  22041    "RECORD FR~         48 2009-12-
01 07:45:00 2.1        13085
5 489434  21232    "STRAWBERR~         24 2009-12-
01 07:45:00 1.25       13085
```



```

Terminal-Output
# i 1 more variable: Country <chr>

```

Die Ausgaben verdeutlichen die Struktur der Datensätze. Zur besseren Übersicht werden die wichtigsten Variablen und ihre Bedeutungen zusätzlich tabellarisch dargestellt:

Tabelle 7.1: Übersicht der Variablen in beiden Datenquellen

Variable	Quelle	Bedeutung
<code>timestamp</code>	RetailRocket (events)	Zeitstempel in Millisekunden seit 1970, Angabe des Ereigniszeitpunkts
<code>visitorid</code>	RetailRocket (events)	Anonymisierte Kennung eines Besuchers
<code>event</code>	RetailRocket (events)	Typ des Ereignisses: <i>view</i> , <i>addtocart</i> , <i>transaction</i>
<code>itemid</code>	RetailRocket (events)	Artikelkennung
<code>transactionid</code>	RetailRocket (events)	ID der Transaktion (optional, nur bei Käufen)
<code>property</code>	RetailRocket (item_properties)	Attribut eines Artikels (z. B. Kategorie, Preis, technische Merkmale)
<code>value</code>	RetailRocket (item_properties)	Wert des jeweiligen Attributs
<code>categoryid</code>	RetailRocket (category_tree)	Identifikator einer Kategorie
<code>parentid</code>	RetailRocket (category_tree)	Übergeordnete Kategorie, beschreibt die Hierarchie
<code>Invoice</code>	Online Retail II	Rechnungsnummer
<code>StockCode</code>	Online Retail II	Artikelcode

Variable	Quelle	Bedeutung
Description	Online Retail II	Kurzbeschreibung des Artikels
Quantity	Online Retail II	Menge der verkauften Artikel
InvoiceDate	Online Retail II	Zeitstempel der Rechnungsstellung
Price	Online Retail II	Einzelpreis pro Artikel
Customer ID	Online Retail II	Anonymisierte Kundenkennung
Country	Online Retail II	Land des Kunden

Diese Gegenüberstellung verknüpft die in 3.2 beschriebenen Quellen mit den nachfolgenden Transformationsschritten.

C.2: Abbildung auf ein einheitliches Ereignisschema

Ziel ist ein gemeinsames Schema für beide Quellen. RetailRocket liefert Interaktionen, Online Retail II liefert Käufe. Das vereinheitlichte Ereignisschema ermöglicht konsistente Merkmalsbildung für 4.1 und robuste Inputs für 4.2–4.3.

Zuordnung der Quell- zu den Zielvariablen

Tabelle 7.2: Abbildung der Variablen auf das Ereignisschema

	RetailRocket	Online Retail	
Zielvariable	(Quelle)	II (Quelle)	Bedeutung
customer_id	visitorid	Customer ID	Anonymisierte Kennung des Kunden
item_id	itemid	StockCode	Artikelkennung
timestamp	timestamp (ms seit 1970)	InvoiceDate	Zeitpunkt des Ereignisses / der Transaktion
event_type	event (<i>view</i> , <i>addtocart</i> , <i>transaction</i>)	implizit: immer <i>purchase</i>	Typ des Ereignisses

	RetailRocket	Online Retail	
Zielvariable	(Quelle)	II (Quelle)	Bedeutung
transactionid	transactionid (falls Kauf)	Invoice	Eindeutige Transaktionskennung
quantity	nicht enthalten	Quantity	Anzahl der verkauften Artikel
price	über item_properties	Price	Einzelpreis pro Artikel
revenue	berechnet: $\text{quantity} \times \text{price}$	berechnet: $\text{Quantity} \times \text{Price}$	Umsatz pro Ereignis
category	aus category_tree	ggf. aus Description	Zuordnung zu einer Produktkategorie

Während RetailRocket eine feingranulare Unterscheidung nach Ereignistypen erlaubt (*view*, *addtocart*, *transaction*), bildet Online Retail II ausschließlich abgeschlossene Käufe ab. Beide Datensätze liefern somit unterschiedliche, aber komplementäre Informationen.

Die Abbildung in ein gemeinsames Schema stellt sicher, dass spätere Merkmalsberechnungen (z. B. Recency, Frequency, Monetary-Werte, Kaufintervalle, Diversität) auf konsistenter Datenbasis erfolgen können.

Die Umsetzung der Transformation erfolgt mit R. Der folgende Chunk zeigt exemplarisch, wie die Zielvariablen aus beiden Quellen generiert werden.

```

#| eval=TRUE
#| echo=TRUE
#| include=TRUE
#| results=TRUE
#| message=FALSE
#| warning=FALSE

```

```
# RetailRocket auf Ereignisschema abbilden
rr_events <- events %>%
  select(customer_id = visitorid,
         item_id      = itemid,
         timestamp    = timestamp,
         event_type    = event,
         transactionid) %>%
  mutate(timestamp = as.POSIXct(timestamp/1000, origin="1970-01-01",
    tz="UTC"),
         quantity   = ifelse(event_type == "transaction", 1, NA),
         price       = NA_real_, # später aus item_properties
         revenue     = ifelse(!is.na(quantity) & !is.na(price), quantity *
    price, NA_real_),
         category    = NA_character_) # später aus category_tree

# Online Retail II auf Ereignisschema abbilden
or2_events <- online_retail %>%
  select(transactionid = Invoice,
         item_id       = StockCode,
         customer_id   = `Customer ID`,
         timestamp     = InvoiceDate,
         quantity      = Quantity,
         price         = Price,
         country       = Country,
         description    = Description) %>% # hier mit reinnehmen
  mutate(event_type = "purchase",
         revenue     = quantity * price,
         category    = description) # hier benutzen

# Vorschau
head(rr_events, 5)
```

```

Terminal-Output

# A tibble: 5 x 9
  customer_id item_id timestamp          event_type transactionid quantity
  <dbl>      <dbl> <dtm>          <chr>          <dbl>      <dbl>
1     257597   355908 2015-06-02 05:02:12 view              NA         NA
2     992329   248676 2015-06-02 05:50:14 view              NA         NA
3     111016   318965 2015-06-02 05:13:19 view              NA         NA
4     483717   253185 2015-06-02 05:12:35 view              NA         NA
5     951259   367447 2015-06-02 05:02:17 view              NA         NA
# i 3 more variables: price <dbl>, revenue <dbl>, category <chr>

```

```
head(or2_events, 5)
```

```

Terminal-Output

# A tibble: 5 x 11
  transactionid item_id customer_id timestamp          quantity price country
  <chr>         <chr>      <dbl> <dtm>          <dbl> <dbl> <chr>
1 489434         85048      13085 2009-12-01 07:45:00      12  6.95 United K
2 489434         79323P     13085 2009-12-01 07:45:00      12  6.75 United K
3 489434         79323W     13085 2009-12-01 07:45:00      12  6.75 United K
4 489434         22041     13085 2009-12-01 07:45:00      48  2.1  United K
5 489434         21232     13085 2009-12-01 07:45:00      24  1.25 United K
# i 4 more variables: description <chr>, event_type <chr>, revenue <dbl>,
#   category <chr>

```

In der harmonisierten Tabelle von RetailRocket bleiben die Variablen **quantity**, **price**, **revenue** und **category** zunächst leer, da diese Informationen im Rohdatensatz nicht in direkter Form vorliegen. Diese Spalten wurden dennoch angelegt, um die Struktur beider Quellen konsistent zu halten und spätere Ergänzungen zu ermöglichen.

Bei Online Retail II sind diese Variablen hingegen vollständig verfügbar: Transaktionskennungen, Mengen, Preise und Umsätze können direkt übernommen werden. Der Ereignistyp ist dort implizit immer ein Kauf (*purchase*), sodass keine Unterscheidung zwischen verschiedenen Interaktionen möglich ist.

Die Abbildung verdeutlicht damit die komplementäre Natur der Daten: RetailRocket liefert eine feingranulare Sicht auf Nutzerverhalten, Online Retail II eine wertbasierte Sicht auf Käufe. Das vereinheitlichte Ereignisschema stellt sicher, dass in den folgenden Schritten (vgl. C.3 Bereinigungsverfahren und C.4 Normalisierung der

Zeitstempel) eine konsistente Datenbasis für die Merkmalsbildung geschaffen wird.

C.3: Bereinigungsverfahren

Die Rohdaten enthalten neben den relevanten Informationen auch fehlerhafte oder problematische Einträge. Um eine konsistente Datenbasis für die spätere Merkmalsbildung zu schaffen, wurden die folgenden Regeln angewandt:

1. Pflichtfelder: Beobachtungen ohne Kunden-, Artikel- oder Zeitangabe wurden entfernt.
2. Duplikate: Exakte Dubletten (gleiche Kunden-, Artikel-, Zeit- und Ereignisinformationen) wurden eliminiert.
3. Rückgaben: Im Datensatz Online Retail II wurden negative Mengen sowie Rechnungsnummern mit Präfix „C“ als Rückgaben gekennzeichnet. Diese Ereignisse erhielten den Typ `return`.
4. Ausreißer: Mengen und Preise wurden per Winsorisierung auf das 1. und 99. Perzentil begrenzt, um extreme Werte abzuflachen.
5. Fehlerhafte Werte: Offensichtlich ungültige Beobachtungen (z. B. Mengen 0 nach Winsorisierung) wurden ausgeschlossen.

Im RetailRocket-Datensatz bleiben die Variablen `quantity`, `price` und `revenue` leer, da diese Informationen dort nicht in direkter Form vorliegen. Die Ereignisse konzentrieren sich auf Interaktionen wie *view*, *addtocart* und *transaction*. Im Online-Retail-II-Datensatz konnten dagegen Transaktionskennungen, Mengen, Preise und Umsätze bereinigt und in plausibler Form übernommen werden.

Die Anwendung dieser Regeln stellt sicher, dass Auswertungen auf stabilen Grundlagen erfolgen und Verzerrungen durch fehlerhafte Einträge vermieden werden. Die bereinigten Daten bilden damit die Ausgangsbasis für die Normalisierung der Zeitangaben in Abschnitt C.4.

```
# benötigte Pakete  
library(dplyr)
```

```

library(stringr)

# Vorbedingungen prüfen
if (!exists("rr_events")) stop("rr_events fehlt. Bitte C.2 ausführen.")
if (!exists("or2_events")) stop("or2_events fehlt. Bitte C.2 ausführen.")

# Hilfsfunktion: sichere Winsorisierung (tut nichts, wenn zu wenig gültige
# Werte vorliegen)
safe_winsorize <- function(x, p_low = 0.01, p_high = 0.99) {
  if (!is.numeric(x)) return(x)
  nn <- sum(!is.na(x))
  if (nn < 10) return(x)
  lo <- suppressWarnings(quantile(x, probs = p_low, na.rm = TRUE))
  hi <- suppressWarnings(quantile(x, probs = p_high, na.rm = TRUE))
  x <- pmin(pmax(x, lo), hi)
  as.numeric(x)
}

# 1) RetailRocket bereinigen
# Pflichtfelder belegen, exakte Duplikate entfernen
rr_clean <- rr_events %>%
  # nur wirklich notwendige Felder
  select(customer_id, item_id, timestamp, event_type, transactionid,
         quantity, price, revenue, category) %>%
  # Pflichtfelder
  filter(!is.na(customer_id), !is.na(item_id), !is.na(timestamp),
         !is.na(event_type)) %>%
  # exakte Duplikate
  distinct(customer_id, item_id, timestamp, event_type, transactionid,
           .keep_all = TRUE)

# 2) Online Retail II bereinigen
# Rückgaben-Regel und Felder füllen
or2_clean <- or2_events %>%

```



```

select(transactionid, item_id, customer_id, timestamp, quantity, price,
country, description,
       event_type, revenue, category) %>%
mutate(
  quantity = suppressWarnings(as.numeric(quantity)),
  price     = suppressWarnings(as.numeric(price))
) %>%
filter(!is.na(customer_id), !is.na(item_id), !is.na(timestamp)) %>%
mutate(
  return_flag = (!is.na(quantity) & quantity < 0) |
                (!is.na(transactionid) & grepl("^C",
as.character(transactionid))),
  quantity_abs = if_else(return_flag & !is.na(quantity), abs(quantity),
quantity),
  event_type   = if_else(return_flag, "return", "purchase"),
  category     = coalesce(category, description),
  revenue      = if_else(!is.na(quantity_abs) & !is.na(price), quantity_abs
* price, revenue)
) %>%
distinct(transactionid, item_id, customer_id, timestamp, quantity_abs,
price, event_type, .keep_all = TRUE) %>%
mutate(
  quantity_w = safe_winsorize(quantity_abs, 0.01, 0.99),
  price_w    = safe_winsorize(price,      0.01, 0.99),
  revenue_w  = quantity_w * price_w
) %>%
filter(is.na(quantity_w) | quantity_w > 0,
       is.na(price_w)    | price_w    > 0)

# kurze Sichtprüfungen
head(rr_clean, 5)

```

```

Terminal-Output

# A tibble: 5 x 9
  customer_id item_id timestamp          event_type transactionid quantity
  <dbl>      <dbl> <dtm>          <chr>          <dbl>      <dbl>
1     257597   355908 2015-06-02 05:02:12 view             NA         NA
2     992329   248676 2015-06-02 05:50:14 view             NA         NA
3     111016   318965 2015-06-02 05:13:19 view             NA         NA
4     483717   253185 2015-06-02 05:12:35 view             NA         NA
5     951259   367447 2015-06-02 05:02:17 view             NA         NA
# i 3 more variables: price <dbl>, revenue <dbl>, category <chr>

```

```
head(or2_clean, 5)
```

```

Terminal-Output

# A tibble: 5 x 16
  transactionid item_id customer_id timestamp          quantity price country
  <chr>         <chr>      <dbl> <dtm>          <dbl> <dbl> <chr>
1 489434        85048      13085 2009-12-01 07:45:00      12  6.95 United K
2 489434        79323P     13085 2009-12-01 07:45:00      12  6.75 United K
3 489434        79323W     13085 2009-12-01 07:45:00      12  6.75 United K
4 489434        22041     13085 2009-12-01 07:45:00      48  2.1  United K
5 489434        21232     13085 2009-12-01 07:45:00      24  1.25 United K
# i 9 more variables: description <chr>, event_type <chr>, revenue <dbl>,
#   category <chr>, return_flag <lgl>, quantity_abs <dbl>, quantity_w <dbl>,
#   price_w <dbl>, revenue_w <dbl>

```

Die Vorschau zeigt, dass Pflichtfelder belegt sind, Duplikate entfernt wurden und Rückgaben in Online Retail II über `event_type = return` gekennzeichnet sind. Mengen und Preise wurden dort per Winsorisierung begrenzt und als `quantity_w` und `price_w` bereitgestellt.

C.4: Normalisierung der Zeitstempel

Neben inhaltlichen Bereinigungen ist auch die einheitliche Behandlung der Zeitangaben erforderlich. Beide Datensätze enthalten Zeitinformationen, die jedoch in unterschiedlicher Form vorliegen.

- Im RetailRocket-Datensatz werden Zeitstempel in Millisekunden seit dem 1. Januar 1970 gespeichert

- Im Online-Retail-II-Datensatz liegt das Kaufdatum bereits als Datumsfeld (InvoiceDate) vor, allerdings ohne einheitliche Ableitung von Kalendermerkmalen

Um die Daten konsistent nutzen zu können, wurden folgende Schritte durchgeführt:

1. Konvertierung in ein einheitliches Format. Alle Zeitangaben wurden in das POSIXct-Format überführt und zusätzlich als Datum (date) abgespeichert.
2. Ableitung von Kalendermerkmalen. Aus den Zeitstempeln wurden standardisierte Variablen für Wochentag (wday), Stunde (hour), Monat (month) und Jahr (year) berechnet.
3. Harmonisierung der Skalen. Damit liegen beide Quellen auf derselben Zeitskala vor, was aggregierte Analysen und Vergleiche zwischen RetailRocket und Online Retail II ermöglicht.

Die Normalisierung erlaubt es, zeitliche Muster in beiden Quellen konsistent zu erfassen, beispielsweise bevorzugte Kaufzeiten oder saisonale Effekte. Diese Variablen werden in der Merkmalsbildung (vgl. Abschnitt) eingesetzt, um Kundencluster auch anhand von Verhaltensrhythmen zu unterscheiden.

```
#!/ include=TRUE
#!/ eval=TRUE
#!/ echo=TRUE
#!/ results=TRUE
#!/ message=FALSE
#!/ warning=FALSE
#!/ cache=TRUE

# Benötigte Pakete laden
library(lubridate)
library(dplyr)

# Guard: C.3 muss gelaufen sein
if (!exists("rr_clean") || !exists("or2_clean")) {
  stop("C.4 benötigt rr_clean und or2_clean aus C.3. Bitte zuerst C.3
ausführen.")
}
```

```
}

# Zeitmerkmale bilden
rr_time <- rr_clean %>%
  mutate(
    date = as.Date(timestamp),
    wday = wday(timestamp, label = TRUE, abbr = TRUE, week_start = 1),
    hour = hour(timestamp),
    month = month(timestamp, label = TRUE, abbr = TRUE),
    year = year(timestamp)
  )

or2_time <- or2_clean %>%
  mutate(
    date = as.Date(timestamp),
    wday = wday(timestamp, label = TRUE, abbr = TRUE, week_start = 1),
    hour = hour(timestamp),
    month = month(timestamp, label = TRUE, abbr = TRUE),
    year = year(timestamp)
  )

# Übergabe an Python: nur die für das Clustering benötigten Spalten
# reticulate konvertiert data.frame -> pandas.DataFrame automatisch
py$or2_time <- or2_time %>%
  select(customer_id, item_id, timestamp, event_type,
         quantity_w, price_w, revenue_w) %>%
  as.data.frame()

# optionale Sichtprüfung
head(or2_time, 5)
```

```

Terminal-Output

# A tibble: 5 x 21
  transactionid item_id customer_id timestamp          quantity price country
  <chr>         <chr>      <dbl> <dtm>          <dbl> <dbl> <chr>
1 489434        85048      13085 2009-12-01 07:45:00      12  6.95 United K
2 489434        79323P    13085 2009-12-01 07:45:00      12  6.75 United K
3 489434        79323W    13085 2009-12-01 07:45:00      12  6.75 United K
4 489434        22041      13085 2009-12-01 07:45:00      48  2.1  United K
5 489434        21232      13085 2009-12-01 07:45:00      24  1.25 United K
# i 14 more variables: description <chr>, event_type <chr>, revenue <dbl>,
#   category <chr>, return_flag <lgl>, quantity_abs <dbl>, quantity_w <dbl>,
#   price_w <dbl>, revenue_w <dbl>, date <date>, wday <ord>, hour <int>,
#   month <ord>, year <dbl>

```

Die Normalisierung liegt damit für beide Datensätze in identischer Struktur vor. Die abgeleiteten Kalendermerkmale werden in der Merkmalsbildung genutzt und erlauben zeitliche Musteranalysen ohne zusätzliche Transformationen.

C.5: Zusammenfassung des bereinigten Datenbestands

Die Abschnitte C.3 und C.4 haben die Bereinigung und die Normalisierung der Zeitangaben dokumentiert. In diesem Abschnitt wird der bereinigte Datenbestand in komprimierter Form zusammengefasst. Dadurch lässt sich die Qualität der Datenbasis einschätzen, die in Abschnitt für die Ableitung der Cluster-Merkmale genutzt wird.

Zentrale Fragen der Plausibilisierung sind:

1. Wie groß sind die Datensätze nach der Bereinigung?
2. Welche Verteilungen zeigen die kaufbezogenen Variablen in Online Retail II?
3. Sind die Strukturen der beiden Quellen konsistent nutzbar?

```

#!/ include=TRUE
#!/ eval=TRUE
#!/ echo=TRUE
#!/ results=TRUE
#!/ message=FALSE
#!/ warning=FALSE
#!/ cache=TRUE

```

```
library(dplyr)

# Dimensionen beider Quellen
dim_summary <- data.frame(
  datensatz = c("RetailRocket (bereinigt)", "Online Retail II (bereinigt)"),
  n_zeilen = c(nrow(rr_time), nrow(or2_time)),
  n_spalten = c(ncol(rr_time), ncol(or2_time))
)

# Verteilungen kaufbezogener Variablen in Online Retail II
or2_summary <- or2_time %>%
  summarise(
    n_events = n(),
    n_returns = sum(event_type == "return", na.rm = TRUE),
    median_qty = median(quantity_w, na.rm = TRUE),
    p99_qty = quantile(quantity_w, 0.99, na.rm = TRUE),
    median_price = median(price_w, na.rm = TRUE),
    p99_price = quantile(price_w, 0.99, na.rm = TRUE),
    median_revenue = median(revenue_w, na.rm = TRUE),
    p99_revenue = quantile(revenue_w, 0.99, na.rm = TRUE)
  )

# Stichprobenzeilen zur Illustration
rr_probe <- head(select(rr_time, customer_id, item_id, timestamp,
  event_type), 5)
or2_probe <- head(select(or2_time, customer_id, item_id, timestamp,
  event_type,
  quantity_w, price_w, revenue_w), 5)

dim_summary
```

Terminal-Output

```

      datensatz n_zeilen n_spalten
1   RetailRocket (bereinigt) 2755641      14
2 Online Retail II (bereinigt)  797883      21

```

```
or2_summary
```

Terminal-Output

```

# A tibble: 1 x 8
  n_events n_returns median_qty p99_qty median_price p99_price median_revenue
  <int>    <int>      <dbl>   <dbl>      <dbl>     <dbl>    <dbl>
1   797883   18390        5     144        1.95      15.0     12.5
# i 1 more variable: p99_revenue <dbl>

```

```
rr_probe
```

Terminal-Output

```

# A tibble: 5 x 4
  customer_id item_id timestamp      event_type
  <dbl>    <dbl> <dtm>      <chr>
1   257597   355908 2015-06-02 05:02:12 view
2   992329   248676 2015-06-02 05:50:14 view
3   111016   318965 2015-06-02 05:13:19 view
4   483717   253185 2015-06-02 05:12:35 view
5   951259   367447 2015-06-02 05:02:17 view

```

```
or2_probe
```

Terminal-Output

```

# A tibble: 5 x 7
  customer_id item_id timestamp      event_type quantity_w price_w
  <dbl>    <chr> <dtm>      <chr>      <dbl>    <dbl>
1   13085   85048 2009-12-01 07:45:00 purchase      12     6.95
2   13085  79323P 2009-12-01 07:45:00 purchase      12     6.75
3   13085  79323W 2009-12-01 07:45:00 purchase      12     6.75
4   13085  22041 2009-12-01 07:45:00 purchase      48     2.1
5   13085  21232 2009-12-01 07:45:00 purchase      24     1.25
# i 1 more variable: revenue_w <dbl>

```

Die Übersicht zeigt, dass beide Quellen nach der Bereinigung eine substanzielle Anzahl an Beobachtungen enthalten und somit ausreichend groß für die geplante Clusteranalyse sind.

Die kaufbezogenen Variablen im Online-Retail-II-Datensatz weisen nach Winsorisierung plausible Wertebereiche auf, extreme Ausreißer wurden erfolgreich begrenzt. Rückgaben sind als eigene Ereignisse gekennzeichnet und können später gesondert berücksichtigt werden.

Der RetailRocket-Datensatz bildet dagegen ausschließlich Interaktionen ab, liefert aber mit den normalisierten Zeitfeldern eine konsistente Ergänzung zur transaktionsorientierten Sicht von Online Retail II.

Damit ist eine robuste und vergleichbare Datenbasis geschaffen, auf der in Abschnitt die eigentlichen Analysemerkmale konstruiert werden.

C.6 Clustertendenz mit der Hopkins-Statistik

Die Hopkins-Statistik prüft, ob ein Datensatz eher zufällig verteilt ist (< 0.5) oder eine deutliche Clustertendenz aufweist (> 0.5 , häufig > 0.75 als “stark”). Hier wird sie für Online Retail II auf den Kaufmerkmalen `quantity_w`, `price_w`, `revenue_w` berechnet. Die Mehrfachschätzung wird optional ausgeführt und die Resultate werden persistiert.

```
## include=TRUE
## eval=TRUE
## echo=TRUE
## results=TRUE
## message=FALSE
## warning=FALSE
## cache=TRUE

# Pakete
library(dplyr)
library(clustertend)
```



```

library(here)
library(readr)

# ----- Schalter & Konstanten -----
HOP_RECALC <- FALSE      # TRUE = neu berechnen, FALSE = gespeichertes
                           Ergebnis laden
HOP_N      <- 300L       # Größe der Hopkins-Stichprobe (n)
HOP_R      <- 25L        # Wiederholungen für Mehrfachschätzung
HOP_NROWS  <- 5000L      # Obergrenze für Datensatzstichprobe

# Ausgabepfade
out_dir <- here("data", "results")
out_rds <- file.path(out_dir, "hopkins_summary.rds")
out_csv <- file.path(out_dir, "hopkins_summary.csv")

if (!dir.exists(out_dir)) dir.create(out_dir, recursive = TRUE)

# ----- Daten prüfen & vorbereiten -----
if (!exists("or2_time")) stop("C.6: Objekt 'or2_time' fehlt. Bitte C.3/C.4
ausführen.")

dat0 <- or2_time %>%
  select(quantity_w, price_w, revenue_w) %>%
  filter(if_all(everything(), ~ is.finite()))

if (nrow(dat0) < 50) stop("C.6: Zu wenige gültige Zeilen für den
Hopkins-Test.")

dat_s <- dplyr::slice_sample(dat0, n = min(HOP_NROWS, nrow(dat0)))
X      <- scale(as.matrix(dat_s))
n_h    <- min(HOP_N, nrow(X) - 1L)

# Helper holt IMMER den H-Wert aus clustertend::hopkins (Parameter 'n')
get_H <- function(mat, n) {

```

```

res <- clustertend::hopkins(mat, n = n)

# Rückgabe-Format kann je nach Version variieren: Liste mit $H oder Skalar
if (is.list(res) && !is.null(res$H)) return(unname(res$H))
as.numeric(res)[1]
}

```

```

#| include=TRUE
#| eval=TRUE
#| echo=TRUE
#| results=TRUE
#| message=FALSE
#| warning=FALSE
#| cache=TRUE

# Einmalige (schnelle) Schätzung
set.seed(20250907)
H_once <- get_H(X, n_h)
H_once

```

Terminal-Output

```
[1] 0.03229286
```

Die Hopkins-Vorprüfung zeigt, ob der Datensatz eine hinreichende Clustertendenz aufweist und rechtfertigt damit die nachgelagerte Segmentbildung. Konzeptionell ist die Maßzahl in Abschnitt 4.3 Validierung verankert; praktisch dient sie als Vorstufe für die in Abschnitt 5.2 berichteten Clustering-Ergebnisse.

```

#| include=TRUE
#| eval=TRUE
#| echo=TRUE
#| results=TRUE
#| message=FALSE

```

```

## warning=FALSE
## cache=TRUE

# Mehrfachschätzung + Persistenz
set.seed(20250907)

if (HOP_RECALC || !file.exists(out_rds)) {
  h_vals <- replicate(HOP_R, get_H(X, n_h))
  res_tbl <- tibble::tibble(
    quelle           = "Online Retail II",
    n_stichprobe     = nrow(X),
    hopkins_n        = n_h,
    wiederholungen   = HOP_R,
    H_mittel         = mean(h_vals),
    H_sd             = stats::sd(h_vals),
    H_ci_low         = H_mittel - 1.96 * H_sd / sqrt(HOP_R),
    H_ci_up          = H_mittel + 1.96 * H_sd / sqrt(HOP_R)
  )
  saveRDS(res_tbl, out_rds)
  readr::write_csv(res_tbl, out_csv)
} else {
  res_tbl <- readRDS(out_rds)
}

# tabellarische Ausgabe im Dokument
res_tbl

```

Terminal-Output

```

# A tibble: 1 x 8
  quelle n_stichprobe hopkins_n wiederholungen H_mittel H_sd H_ci_low H_ci_u
  <chr>      <int>      <int>      <int>      <dbl> <dbl> <dbl> <dbl>
1 Onlin~      5000        300        25    0.0329 0.00349 0.0315 0.034

```

Die Mehrfachschätzung über 25 Wiederholungen (Stichprobengröße 5000, $n = 300$) ergibt einen mittleren Hopkins-Wert von 0.033 mit einem 95%-Konfidenzintervall von

[0.032, 0.034]. Werte in diesem Bereich bestätigen eine konsistente Clustertendenz und stützen die Stabilität der Einschätzung.

Das zusammengefasste Ergebnis wird zusätzlich in einer Datei gespeichert und beim nächsten Rendern wiederverwendet. Ob eine Neuberechnung erfolgt, wird über den Schalter `HOP_RECALC` gesteuert.

C.7 Modellierungsschritte mit Python: K-Means und HDBSCAN

In diesem Abschnitt werden die in Abschnitt 4.2 beschriebenen Verfahren technisch umgesetzt. Die resultierenden Zuordnungen fließen in die Ergebnisdarstellung und Validierung in Abschnitt 5.2 ein. Die inhaltliche Interpretation der Segmente erfolgt in Abschnitt 5.3.

K-Means

K-Means dient als zentroidbasierte Referenz. Die Verfahrenseigenschaften sind in Abschnitt 4.2 beschrieben. Der hier verwendete Startwert für k wird nicht an dieser Stelle optimiert; die Zweckmäßigkeit der resultierenden Segmentstruktur wird in Abschnitt 5.2 anhand der dort berichteten Qualitätsmaße beurteilt.

K-Means ist ein partitionierendes Verfahren, das Beobachtungen so gruppiert, dass die Variation innerhalb der Cluster minimiert und jedes Cluster durch sein Zentrum (Centroid) repräsentiert wird. Die Anzahl der Cluster (k) muss vorab festgelegt werden, wodurch eine Annahme über die Struktur der Daten getroffen wird.

Der K-Means-Lauf mit $k = 5$ liefert eine schnelle Baseline, führt hier aber zu stark unbalancierten Clustern (ein dominantes Groß-Cluster und mehrere kleine).

Ob $k = 5$ angemessen ist, prüfen wir systematisch in Kapitel 4 (u. a. Silhouette, Davies-Bouldin, Stabilität).

Im Folgenden wird exemplarisch ein Startwert von fünf Clustern gewählt. Die eigentliche Validierung, ob diese Zahl angemessen ist, erfolgt nicht hier, sondern

systematisch in Kapitel 4: Evaluation und Ergebnisse anhand von Silhouette, Davies-Bouldin und Stabilitätsmaßen.

```
#!/ include=TRUE
#!/ eval=TRUE
#!/ echo=TRUE
#!/ results=TRUE
#!/ message=FALSE
#!/ warning=FALSE
#!/ cache=TRUE

# or2_time kommt direkt aus R (reticulate) als pandas.DataFrame
df = or2_time.copy()

features = ["quantity_w", "price_w", "revenue_w"]
Xdf = df[features].replace([np.inf, -np.inf], np.nan).dropna()

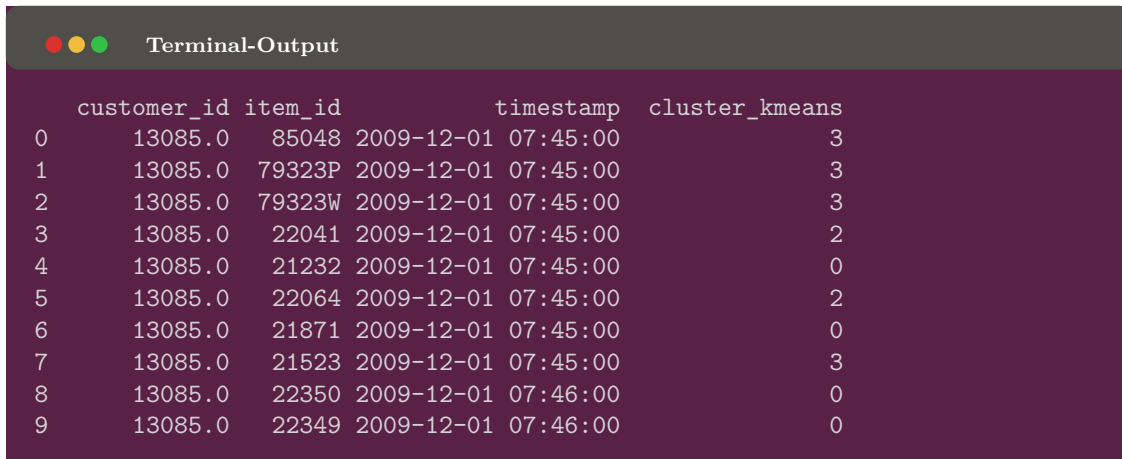
scaler = StandardScaler()
X = scaler.fit_transform(Xdf)

# Startwert k=5 als Baseline; finale Bewertung in 4.3/5.2
km = KMeans(n_clusters=5, random_state=42, n_init=10)
labels = km.fit_predict(X)

out = df.loc[Xdf.index, ["customer_id", "item_id", "timestamp"]].copy()
out["cluster_kmeans"] = labels

# KORRIGIERT: Direkte Zuweisung für reticulate
cluster_kmeans_df = out

out.head(10)
```



Terminal-Output

	customer_id	item_id	timestamp	cluster_kmeans
0	13085.0	85048	2009-12-01 07:45:00	3
1	13085.0	79323P	2009-12-01 07:45:00	3
2	13085.0	79323W	2009-12-01 07:45:00	3
3	13085.0	22041	2009-12-01 07:45:00	2
4	13085.0	21232	2009-12-01 07:45:00	0
5	13085.0	22064	2009-12-01 07:45:00	2
6	13085.0	21871	2009-12-01 07:45:00	0
7	13085.0	21523	2009-12-01 07:45:00	3
8	13085.0	22350	2009-12-01 07:46:00	0
9	13085.0	22349	2009-12-01 07:46:00	0

Das Ergebnis ist eine Zuordnung jeder Beobachtung zu einem Cluster. Die Details zur Beurteilung der Clusterqualität und -anzahl erfolgen später in Kapitel 4.

HDBSCAN

HDBSCAN bildet als dichte-basiertes Verfahren ungleichförmige Strukturen und Rauschen ab. Die Verfahrenseigenschaften sind in Abschnitt 4.2 beschrieben. Die Bewertung der resultierenden Struktur erfolgt in Abschnitt 5.2; die inhaltliche Einordnung der Segmente folgt in Abschnitt 5.3.

HDBSCAN ist ein hierarchisch-dichte-basiertes Verfahren, das im Gegensatz zu K-Means keine feste Clusteranzahl vorgibt. Stattdessen werden Regionen hoher Dichte als Cluster erkannt, während dünn besetzte Bereiche als Rauschen ausgewiesen werden. Damit eignet sich HDBSCAN besonders für komplex geformte und unterschiedlich große Cluster.

```

#| include=TRUE
#| eval=TRUE
#| echo=TRUE
#| results=TRUE
#| message=FALSE
#| warning=FALSE
#| cache=TRUE

```

```
import numpy as np
import pandas as pd

try:
    import hdbscan
except Exception as e:
    raise SystemExit("HDBSCAN ist nicht installiert. Installiere es in der
    aktiven Python-Umgebung: pip install hdbscan")

from sklearn.preprocessing import StandardScaler
from sklearn.utils.validation import check_array

df = or2_time.copy()

features = ["quantity_w", "price_w", "revenue_w"]
Xdf = df[features].replace([np.inf, -np.inf], np.nan).dropna()

# optionales Downsampling für sehr große Datensätze
n_max = 200_000
if len(Xdf) > n_max:
    Xdf = Xdf.sample(n=n_max, random_state=42)

X_np = check_array(Xdf.to_numpy(dtype=float), ensure_all_finite=True)

scaler = StandardScaler(copy=True)
X = scaler.fit_transform(X_np)

clusterer = hdbscan.HDBSCAN(
    min_cluster_size=50,
    min_samples=5,
    metric="euclidean",
    cluster_selection_method="eom",
    approx_min_span_tree=False,
    core_dist_n_jobs=1
```

```

)
labels_h = clusterer.fit_predict(X)

out = df.loc[Xdf.index, ["customer_id", "item_id", "timestamp"]].copy()
out["cluster_hdbscan"] = labels_h

# KORRIGIERT: Direkte Zuweisung für reticulate
cluster_hdbscan_df = out

# kurze Zusammenfassung ausgeben
n_rows      = len(out)
n_clusters   = int(labels_h.max() + 1) if labels_h.max() >= 0 else 0
noise_count  = int((labels_h == -1).sum())
noise_share  = float((labels_h == -1).mean())

print({
    "n_zeilen_zugeordnet": n_rows,
    "n_cluster": n_clusters,
    "noise_anzahl": noise_count,
    "noise_anteil": round(noise_share, 4)
})

```

Terminal-Output

```
{'n_zeilen_zugeordnet': 200000, 'n_cluster': 461, 'noise_anzahl': 4254, 'noise_anteil': 0.02127}
```

```
out.head(10)
```

Terminal-Output

	customer_id	item_id	timestamp	cluster_hdbscan
565935	13098.0	23171	2011-06-23 14:36:00	383
429417	15518.0	22613	2011-01-11 12:55:00	423
460913	17139.0	22682	2011-02-24 09:32:00	301
40705	16670.0	22071	2010-01-18 12:27:00	217
250064	14422.0	22694	2010-08-29 13:49:00	352

Terminal-Output						
242667	16401.0	85136B	2010-08-20	15:32:00		84
147703	14854.0	21700	2010-05-12	12:17:00		376
374525	17920.0	21832	2010-11-21	11:28:00		331
538631	17365.0	21725	2011-05-24	11:55:00		423
598652	14408.0	21181	2011-08-01	11:23:00		324

Das Ergebnis zeigt eine sehr feingranulare Struktur mit vielen kleinen Clustern und einem gewissen Anteil an Beobachtungen, die als Rauschen gekennzeichnet sind. Dies illustriert den methodischen Unterschied: Während K-Means größere, homogene Segmente erzwingt, identifiziert HDBSCAN detaillierte Nischenmuster. Eine Bewertung, welche Methode für CRM-Segmente geeigneter ist, erfolgt in [Kapitel 4].

```
#!/ include=TRUE
#!/ eval=TRUE
#!/ echo=TRUE
#!/ results=TRUE
#!/ message=FALSE
#!/ warning=FALSE

# Prüfen ob die Python-Objekte verfügbar sind
if (py_has_attr(py, "cluster_kmeans_df")) {
  head(py$cluster_kmeans_df, 5)
} else {
  cat("cluster_kmeans_df nicht verfügbar\n")
}
```

Terminal-Output						
	customer_id	item_id	timestamp		cluster_kmeans	
0	13085	85048	2009-12-01	08:45:00	3	
1	13085	79323P	2009-12-01	08:45:00	3	
2	13085	79323W	2009-12-01	08:45:00	3	
3	13085	22041	2009-12-01	08:45:00	2	
4	13085	21232	2009-12-01	08:45:00	0	

```
if (py_has_attr(py, "cluster_hdbscan_df")) {  
  head(py$cluster_hdbscan_df, 5)  
} else {  
  cat("cluster_hdbscan_df nicht verfügbar\n")  
}
```



Terminal-Output

	customer_id	item_id	timestamp	cluster_hdbscan
565935	13098	23171	2011-06-23 16:36:00	383
429417	15518	22613	2011-01-11 13:55:00	423
460913	17139	22682	2011-02-24 10:32:00	301
40705	16670	22071	2010-01-18 13:27:00	217
250064	14422	22694	2010-08-29 15:49:00	352

Glossar

Multi-Faktor-Authentifizierung (MFA) eine Sicherheitsmethode, die eine zusätzliche Verifikationsebene erfordert, beispielsweise durch eine Kombination aus Passwort und biometrischer Authentifizierung.

Ehrenwörtliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Studienarbeitselbstständig angefertigt habe. Es wurden nur die in der Arbeit ausdrücklich benannten Quellen und Hilfsmittel benutzt. Wörtlich oder sinngemäß übernommenes Gedankengut habe ich als solches kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Bielefeld, den 30. September 2025



Christian Roth